Algorithms: Design and Analysis (Part II)

Mihir Rao

Contents

1 Introduction: Two Motivating Applications **4**
1.1 Distributed shortest-path routing . . . 4
1.2 Sequence alignment . . . 4

2 Selected Review from Part I **6**
2.1 Integer multiplication and the designer's mantra . . . 6
2.2 Guiding principles for analysis . . . 6
2.3 Big-O notation . . . 7
2.4 Graphs and their representations . . . 7
2.5 Graph search . . . 8
2.6 Dijkstra's shortest-path algorithm . . . 9
2.7 Data structures and heaps . . . 10

3 Introduction to Greedy Algorithms **12**
3.1 The greedy paradigm . . . 12
3.2 Proving greedy algorithms correct . . . 12
3.3 Application: optimal caching . . . 13

4 A Scheduling Application **14**
4.1 Problem definition . . . 14
4.2 Designing the greedy algorithm . . . 14
4.3 Correctness with distinct ratios . . . 15
4.4 Handling ties . . . 16

5 Prim's Minimum Spanning Tree Algorithm **17**
5.1 The minimum spanning tree problem . . . 17
5.2 Prim's algorithm . . . 18
5.3 Cuts and three graph facts . . . 18
5.4 Part I: Prim's algorithm outputs a spanning tree . . . 19
5.5 Part II: the cut property . . . 19
5.6 Proof of the cut property . . . 20
5.7 A fast implementation with heaps . . . 21

6 Kruskal's Minimum Spanning Tree Algorithm **23**
6.1 Kruskal's algorithm . . . 23
6.2 Correctness . . . 24
6.3 Running time of the straightforward implementation . . . 25
6.4 The union-find data structure . . . 25
6.5 State of the art and open questions . . . 26

7 Clustering **28**
7.1 Max-spacing k-clusterings . . . 28
7.2 Single-link clustering . . . 28

7.3 Correctness . . . 29

8 Advanced Union-Find **31**
8.1 Lazy unions . . . 31
8.2 Union by rank . . . 32
8.3 Path compression . . . 34
8.4 The Hopcroft-Ullman bound . . . 35
8.5 The Ackermann function . . . 36
8.6 Tarjan's bound . . . 37

9 Huffman Codes **40**
9.1 Prefix-free codes . . . 40
9.2 Codes as trees . . . 40
9.3 Huffman's greedy algorithm . . . 41
9.4 Correctness . . . 43
9.5 Running time . . . 44

10 Introduction to Dynamic Programming **45**
10.1 Weighted independent sets in path graphs . . . 45
10.2 Optimal substructure . . . 46
10.3 Eliminating redundant recursion . . . 46
10.4 Reconstructing an optimal solution . . . 47
10.5 Principles of dynamic programming . . . 48

11 The Knapsack Problem **49**
11.1 Optimal substructure . . . 49
11.2 The recurrence and the subproblems . . . 49
11.3 An example . . . 50

12 Sequence Alignment **52**
12.1 Problem definition . . . 52
12.2 Optimal substructure . . . 52
12.3 Subproblems and recurrence . . . 53
12.4 The algorithm . . . 54
12.5 Reconstructing an optimal alignment . . . 54

13 Optimal Binary Search Trees **55**
13.1 Search trees and non-uniform access . . . 55
13.2 Problem definition . . . 55
13.3 Greedy does not work . . . 56
13.4 Optimal substructure . . . 57
13.5 Subproblems and recurrence . . . 57
13.6 The algorithm . . . 58

14 The Bellman-Ford Algorithm **60**
14.1 Negative cycles . . . 60
14.2 Optimal substructure . . . 61
14.3 The recurrence and the algorithm . . . 62
14.4 Detecting negative cycles . . . 63
14.5 Space and predecessor pointers . . . 63
14.6 From Bellman-Ford to Internet routing . . . 64

15 All-Pairs Shortest Paths **65**
15.1 Reduction to single-source shortest paths . . . 65
15.2 Optimal substructure . . . 65

15.3 The Floyd-Warshall algorithm . . . 66
15.4 Reweighting . . . 67
15.5 Johnson's algorithm . . . 68

16 NP-Complete Problems **70**
16.1 Polynomial-time solvability and the class P . . . 70
16.2 Reductions and completeness . . . 71
16.3 The class NP and NP-completeness . . . 71
16.4 The P versus NP question . . . 72
16.5 Algorithmic approaches to NP-complete problems . . . 73

17 Faster Exact Algorithms for NP-Complete Problems **74**
17.1 The vertex cover problem . . . 74
17.2 Smarter search for small vertex covers . . . 75
17.3 The traveling salesman problem . . . 76
17.4 A dynamic programming algorithm for TSP . . . 77

18 Approximation Algorithms for NP-Complete Problems **79**
18.1 Knapsack and a greedy heuristic . . . 79
18.2 Analysis of the greedy heuristic . . . 80
18.3 A dynamic programming heuristic . . . 81
18.4 Dynamic programming for knapsack, revisited . . . 82
18.5 Analysis of the dynamic programming heuristic . . . 83

19 Local Search Algorithms **84**
19.1 The maximum cut problem . . . 84
19.2 Principles of local search . . . 86
19.3 The 2-SAT problem . . . 87
19.4 Random walks on a line . . . 87
19.5 Analysis of Papadimitriou's algorithm . . . 89

20 The Wider World of Algorithms **90**
20.1 Stable matchings . . . 90
20.2 Bipartite matching and maximum flow . . . 91
20.3 Selfish flow and Braess's paradox . . . 92
20.4 Linear programming . . . 93
20.5 Other topics and models . . . 93

Lecture 1: Introduction: Two Motivating Applications

Two applications preview the main themes of the course: a shortest-path problem that must be solved with only local computation, and a similarity measure that is useful only because an efficient algorithm computes it.

1.1 Distributed shortest-path routing

Many networks are naturally modeled as graphs.

- The Internet: vertices are end hosts and routers; directed edges are direct physical or wireless connections.
- The web graph: vertices are web pages; directed edges are hyperlinks.
- Social networks: vertices are people; edges are friend or follow relationships.

Suppose one gateway router must send data to another gateway router over multiple hops. A natural choice of route is a shortest one, for example the one with the fewest hops. This calls for a shortest-path algorithm. Dijkstra's algorithm computes shortest paths when all edge lengths are nonnegative, but it processes the graph centrally: the source router would need to know the entire Internet. Routing therefore needs a shortest-path algorithm that uses only *local* computation, in which each router works with information from its neighbors. The Bellman–Ford algorithm has this property, and it also handles negative edge costs.

1.2 Sequence alignment

Sequence alignment is a fundamental problem in computational genomics. The input is two strings over the alphabet $\{A, C, G, T\}$, representing portions of one or more genomes, and the goal is to measure how similar they are. Applications include extrapolating the function of genome substrings and estimating proximity in an evolutionary tree, since similar genomes can indicate close relatives.

The strings $AGGGCT$ and $AGGCA$ are intuitively similar because they can be aligned well. An alignment inserts gaps into the two strings so that they have equal length, and then compares them column by column. One alignment of the example is

$$\begin{array}{cccccc} A & G & G & G & C & T \\ A & G & G & - & C & A \end{array}$$

It uses one gap and one mismatch, between T and A. Similarity is measured by the quality of the best alignment, using experimentally determined penalties.

Definition 1.1 [Needleman–Wunsch score]. The input is two strings over $\{A, C, G, T\}$, a penalty $\text{pen}_{\text{gap}} \geq 0$ for each gap, and a penalty $\text{pen}_{xy} \geq 0$ for each column that mismatches characters x and y (for example pen_{AT}). The total penalty of an alignment is the sum of its gap and mismatch penalties. An alignment minimizing the total penalty is the goal; its total penalty is the Needleman–Wunsch score.

A small score indicates similar strings. In the example above, the displayed alignment has penalty $\text{pen}_{\text{gap}} + \text{pen}_{AT}$.

The measure is useful only with an efficient algorithm for finding a best alignment. Brute-force search over all alignments is hopeless: for two strings of length 500, there are more than $2^{500} \geq 10^{125}$ alignments, far more than the roughly 10^{80} atoms in the known universe. Indeed, aligning every character of each string against a gap already gives one distinct alignment for each interleaving of the two strings, and there are $\binom{1000}{500} \geq 2^{500}$ interleavings. The problem instead has a straightforward dynamic-programming solution, developed later in the course.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Introduction: Two Motivating Applications.

Lecture 2: Selected Review from Part I

This unit recalls the parts of Part I that Part II relies on: the vocabulary for analyzing algorithms, graphs and graph search, Dijkstra's shortest-path algorithm, and heaps.

2.1 Integer multiplication and the designer's mantra

Given two n-digit numbers x and y, compute $x \cdot y$. A primitive operation adds or multiplies two single-digit numbers. The grade-school algorithm forms n shifted partial products, each with roughly n operations up to a constant, and adds them; the total is a constant times n^2 operations.

The algorithm designer should refuse to be content with the obvious method and ask whether it can be improved. Write

$$x = 10^{n/2}a + b, \qquad y = 10^{n/2}c + d,$$

where a, b, c, d are $n/2$-digit numbers; for $x = 5678$ and $y = 1234$, these are $a = 56$, $b = 78$, $c = 12$, $d = 34$. Then

$$x \cdot y = 10^n ac + 10^{n/2}(ad + bc) + bd. \tag{2.1}$$

A first recursive algorithm computes ac, ad, bc, bd recursively and then evaluates equation (2.1) directly, with a simple base case for single digits. Karatsuba multiplication needs only three recursive products: compute ac, bd, and $(a+b)(c+d) = ac + ad + bc + bd$ recursively. Gauss's trick recovers the middle coefficient as $ad + bc = (a+b)(c+d) - ac - bd$, so three recursive multiplications and some additions suffice. Which of these algorithms is fastest is a question for the analysis tools below.

2.2 Guiding principles for analysis

1. Worst-case analysis: the running-time bound holds for *every* input of length n. This is particularly appropriate for general-purpose routines. Average-case analysis and benchmarks, by contrast, require domain knowledge about which inputs are likely. Worst cases are also usually easier to analyze.
2. Pay little attention to constant factors and lower-order terms. This is much easier, the constants depend on the architecture, compiler, and programmer anyway, and very little predictive power is lost.
3. Asymptotic analysis: focus on the running time for large input sizes n, since only big problems are interesting.

For example, $6n \log_2 n + 6n$ (merge sort) is "better than" $\frac{1}{2}n^2$ (roughly insertion sort). The quadratic

is smaller for small n, but the two expressions are equal near $n \approx 90$, and at $n = 1400$ the quadratic is already about ten times larger. Taken together, the three principles give a working definition: a FAST ALGORITHM is one whose worst-case running time grows slowly with the input size. Usually the goal is a running time as close to linear, $O(n)$, as possible.

2.3 Big-O notation

Let $T(n)$ be a function on $n = 1, 2, 3, \ldots$, usually the worst-case running time of an algorithm. Informally, $T(n) = O(f(n))$ when, for all sufficiently large n, $T(n)$ is bounded above by a constant multiple of $f(n)$.

Definition 2.1 [BIG-O]. $T(n) = O(f(n))$ if and only if there exist constants $c, n_0 > 0$ such that

$$T(n) \leq c \cdot f(n) \qquad \text{for all } n \geq n_0.$$

The constants c and n_0 cannot depend on n. In a picture, T may exceed f, and even cf for small n, but beyond n_0 the curve T stays below cf (for instance, below $2f$ with $c = 2$).

2.4 Graphs and their representations

A GRAPH has two ingredients: VERTICES, or nodes, V, and EDGES E, which are pairs of vertices. An edge is either UNDIRECTED, an unordered pair, or DIRECTED, an ordered pair (also called an ARC). Examples include road networks, the web, social networks, and precedence constraints. Throughout, $n = |V|$ and $m = |E|$.

An undirected graph with no parallel edges that is connected ("in one piece") has between $n - 1$ and $n(n-1)/2$ edges: a tree attains the minimum, and including every pair of vertices attains the maximum. In most, but not all, applications m is $\Omega(n)$ and $O(n^2)$. In a SPARSE GRAPH, m is $O(n)$ or close to it; in a DENSE GRAPH, m is closer to $\Theta(n^2)$.

The ADJACENCY MATRIX represents G by an $n \times n$ 0–1 matrix A, with $A_{ij} = 1$ exactly when G has an i–j edge. Variants let A_{ij} count parallel i–j edges, store the weight of the i–j edge, or record direction with $A_{ij} = +1$ for an edge from i to j and $A_{ij} = -1$ for an edge from j to i. It uses $\Theta(n^2)$ space, regardless of m.

Adjacency lists have four ingredients, with the following space requirements.

Ingredient	Space
Array (or list) of vertices	$\Theta(n)$
Array (or list) of edges	$\Theta(m)$
Each edge points to its endpoints	$\Theta(m)$
Each vertex points to the edges incident on it	$\Theta(m)$
Total	$\Theta(m+n)$

The last two items are in one-to-one correspondence: each edge-endpoint pointer matches a vertex-edge pointer, so both have size $\Theta(m)$. The total is also written $\Theta(\max\{m, n\})$. Which representation is better depends on the graph density and the operations needed; this course focuses on adjacency lists.

2.5 Graph search

Graph search answers questions such as: is a network connected, so that every vertex can reach every other? It also underlies driving directions, formulating a plan (for a Sudoku puzzle, nodes are partially completed puzzles and arcs fill in one new square), and computing the pieces, or connected components, of a graph, used in clustering and in studying the structure of the web graph. The goals are to find everything findable from a given start vertex and to explore nothing twice, aiming for $O(m+n)$ time.

Algorithm 2.1: Generic graph search

Input: Graph $G = (V, E)$, undirected or directed, and a start vertex s.
Output: The vertices marked explored.

```
1 Mark s explored and all other vertices unexplored
2 while some edge (u, v) has u explored and v unexplored do
3     Choose such an edge (u, v)
4     Mark v explored
```

Proposition 2.2 [Generic search finds exactly the reachable vertices]. *At the end of generic graph search, v is explored if and only if G has a path from s to v.*

Proof. ($\Rightarrow$) By induction on the number of iterations: s is reachable, and a vertex v newly explored through an edge (u, v) with u explored is reachable by extending a path to u by that edge.
($\Leftarrow$) Suppose G has a path P from s to v, but v is unexplored at the end. Since s is explored, some edge (u, w) of P has u explored and w unexplored. Then the loop condition still holds, so the algorithm would not have terminated, a contradiction. □

The generic algorithm leaves open how to choose among the possibly many frontier edges, those

crossing from explored to unexplored vertices.

- Breadth-first search (BFS) explores nodes in layers, using a queue (first in, first out) and $O(m + n)$ time. It computes shortest paths (when every edge counts as length one) and the connected components of an undirected graph.
- Depth-first search (DFS) explores aggressively, like a maze, backtracking only when necessary. It uses a stack (last in, first out), or recursion, and $O(m + n)$ time. It computes a topological ordering of a directed acyclic graph and the connected components of directed graphs.

2.6 Dijkstra's shortest-path algorithm

Definition 2.3 [Single-source shortest paths]. The input is a directed graph $G = (V, E)$ with a nonnegative length ℓ_e on each edge e and a source vertex s. The length of a path is the sum of its edge lengths. The output is, for each $v \in V$, the length $L(v)$ of a shortest s–v path in G.

For convenience, assume that every vertex is reachable from s. The important assumption is $\ell_e \geq 0$ for every $e \in E$. Breadth-first search already computes shortest paths in linear time if $\ell_e = 1$ for every edge. Replacing each edge e by a directed path of ℓ_e unit-length edges would reduce the general case to that one, but this blows up the graph too much. Dijkstra's algorithm solves the problem directly.

Algorithm 2.2: Dijkstra's algorithm

Input: Directed graph $G = (V, E)$, lengths $\ell_e \geq 0$, source s.
Output: Shortest-path distances $A[v]$ and paths $B[v]$ for all $v \in V$.

$X \leftarrow \{s\}$ *// vertices processed so far*
$A[s] \leftarrow 0$; $B[s] \leftarrow$ the empty path
while $X \neq V$ **do**
    Among all edges $(v, w) \in E$ with $v \in X$ and $w \notin X$, choose (v^*, w^*) minimizing $A[v] + \ell_{vw}$
    Add w^* to X
    $A[w^*] \leftarrow A[v^*] + \ell_{v^*w^*}$
    $B[w^*] \leftarrow B[v^*] \cup (v^*, w^*)$

The quantity $A[v] + \ell_{vw}$ is Dijkstra's greedy criterion; the value $A[v]$ was computed in an earlier iteration. Each iteration grows X by one vertex. The array B of paths serves only the explanation; an implementation need not store it.

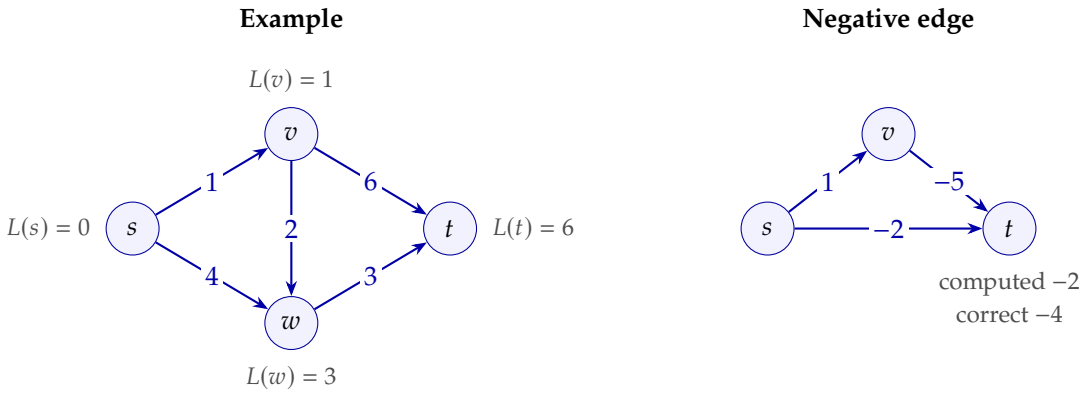


Figure 1: Left: Dijkstra's algorithm on the example processes s, v, w, t in that order and returns $L(s), L(v), L(w), L(t) = 0, 1, 3, 6$; the shortest s–t path is $s \to v \to w \to t$. Right: with a negative edge, the algorithm reports distance -2 to t, while the true shortest distance is -4.

On the example, the first iteration compares the scores $0 + 1$ and $0 + 4$ and adds v with $A[v] = 1$. The second compares the scores 4, $1 + 2 = 3$, and $1 + 6 = 7$, adding w with $A[w] = 3$ and $B[w] = s \to v \to w$. The third compares $3 + 3 = 6$ with 7, adding t with $A[t] = 6$ and $B[t] = s \to v \to w \to t$.

Negative edge lengths cannot be removed by adding a large constant to every edge, because that does not preserve shortest paths: it penalizes paths with more edges. In the right-hand graph, adding 5 to each length makes $s \to t$ (length 3) shorter than $s \to v \to t$ (length 6), whereas originally $s \to v \to t$ is shorter. Dijkstra's algorithm itself is also incorrect on this graph. Its first iteration compares the scores $0 + 1$ for v and $0 + (-2)$ for t, so it adds t with $A[t] = -2$, rather than the correct distance $1 + (-5) = -4$.

2.7 Data structures and heaps

A DATA STRUCTURE organizes data so that it can be accessed quickly and usefully. Examples include lists, stacks, queues, heaps, search trees, hash tables, Bloom filters, and union-find. Different data structures support different sets of operations, so they suit different tasks. Rule of thumb: choose the "minimal" data structure that supports all the operations you need. The level of fluency to aim for is to recognize when a problem calls out for a particular data structure, such as a heap.

A HEAP is a container for objects that have keys, such as employee records, network edges, or events. With n the number of objects in the heap, it supports:

- INSERT: add a new object, in $O(\log n)$ time;
- EXTRACT-MIN: remove an object with a minimum key value, ties broken arbitrarily, in $O(\log n)$ time;
- HEAPIFY: n batched inserts in $O(n)$ time;

- DELETE: in $O(\log n)$ time.

A heap can equally well support EXTRACT-MAX instead of EXTRACT-MIN. The term PRIORITY QUEUE is a synonym for a heap.

Sorting. The canonical use of a heap is fast repeated minimum computations. Selection sort performs $\Theta(n)$ linear scans, taking $\Theta(n^2)$ time on an array of length n. HEAPSORT inserts all n array elements into a heap and then calls EXTRACT-MIN n times, plucking out the elements in sorted order. This is $2n$ heap operations, so $O(n \log n)$ time, which is optimal for a comparison-based sorting algorithm.

Event manager. In a simulation, for example of a video game, the objects are event records (actions or updates to occur at given future times), and the key is the time the event is scheduled to occur. EXTRACT-MIN yields the next scheduled event.

Median maintenance. Numbers $x_1, \ldots, x_n$ arrive one by one, and at each time step i we must report the median of $\{x_1, \ldots, x_i\}$ using $O(\log i)$ time per step. Maintain two heaps: H_{low}, supporting EXTRACT-MAX, and H_{high}, supporting EXTRACT-MIN. The key idea is the invariant that H_{low} holds the $\approx i/2$ smallest elements seen so far and H_{high} the $\approx i/2$ largest. To insert x_i, put it in H_{low} if that heap is empty or x_i is at most its maximum, and in H_{high} otherwise. If the sizes then differ by two, extract the root of the larger heap and insert it into the smaller one. This takes $O(1)$ heap operations, so $O(\log i)$ time. Given the invariant, the median is the root of the larger heap, or the root of H_{low} (the $(i/2)$th smallest element) when the sizes are equal; extracting and reinserting that root finds it in $O(\log i)$ time.

Speeding up Dijkstra. A naive implementation of Dijkstra's algorithm runs in $\Theta(nm)$ time: about n loop iterations, each a linear scan through the edges for the minimum computation. Using heaps reduces the running time to $O(m \log n)$.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Selected Review from Part I.

Lecture 3: Introduction to Greedy Algorithms

There is no single "silver bullet" for algorithm design. The main design paradigms of the two courses are divide and conquer (Part I), randomized algorithms (touched on in Part I), greedy algorithms (next), and dynamic programming (later in Part II).

3.1 The greedy paradigm

Informally, a GREEDY ALGORITHM iteratively makes "myopic" decisions and hopes that everything works out at the end. Dijkstra's shortest-path algorithm is an example: it processes each destination once, irrevocably.

Compared with divide and conquer:

1. It is easy to propose multiple greedy algorithms for many problems.
2. Running-time analysis is usually easy, in contrast with tools such as the master method.
3. Correctness is hard to establish, in contrast with the usually straightforward inductive correctness proofs for divide and conquer.

Most greedy algorithms are *not* correct, even when intuition says otherwise.

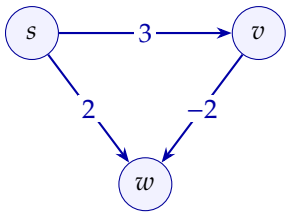


Figure 2: Dijkstra's algorithm with a negative edge length. It reports 2 as the shortest s–w distance; the correct answer is 1.

Example: Run Dijkstra's algorithm from s on the graph above. The first iteration compares the scores $0 + 3$ for v and $0 + 2$ for w, so it adds w with distance 2 and never revisits it. The path $s \to v \to w$, however, has length $3 + (-2) = 1$. The greedy choice fails once edge lengths can be negative.

3.2 Proving greedy algorithms correct

1. INDUCTION, often in the form "greedy stays ahead." Part I's correctness proof for Dijkstra's algorithm is an example.

2. EXCHANGE ARGUMENTS: transform another solution into the greedy one by exchanges that do not make it worse. The scheduling application of Lecture 4 is an example.
3. Whatever works.

3.3 Application: optimal caching

A small fast memory, the CACHE, sits in front of a big slow memory. A sequence of PAGE REQUESTS is processed in order. A request for a page that is not in the cache is a FAULT, or cache miss; the page must then be brought in, and when the cache is full, something must be evicted to make room. The question is what to evict.

Example: The cache holds a, b, c, d, and the request sequence is c, d, e, f, a, b. Suppose the faults on e and f evict a and b. Then the later requests for a and b also fault, for 4 page faults in total. Two faults, on e and f, were inevitable. The other two are consequences of poor eviction choices: evicting c and d, which are never requested again, would have left only 2 faults.

The FURTHEST-IN-FUTURE algorithm, on each fault with a full cache, evicts the cached page whose next request is furthest in the future (a page never requested again counts as furthest). In the example, it evicts c and d.

Theorem 3.1 [BÉLÁDY, 1960s**].** *The furthest-in-future algorithm is optimal: it minimizes the number of cache misses.*

The algorithm needs the future request sequence, so it is not directly implementable. It is still useful in two ways. First, it guides practical algorithms: for example, evicting the least recently used page (LRU) should do well when the data exhibit locality of reference, since then the recent past predicts the near future. Second, it is an idealized benchmark for caching algorithms. The proof is a tricky exchange argument and is not covered here; finding a simple proof remains an open question.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Introduction to Greedy Algorithms.

Lecture 4: A Scheduling Application

One shared resource, such as a processor, must perform many jobs, such as processes. In what order should the jobs be sequenced? This problem gives a first complete example of designing a greedy algorithm, ruling out a plausible alternative, and proving correctness by an exchange argument.

4.1 Problem definition

Each job j has a WEIGHT w_j, its priority, and a LENGTH $\ell_j > 0$. A schedule is an ordering of the n jobs, processed one after another starting at time 0.

Definition 4.1 [COMPLETION TIME]. The COMPLETION TIME C_j of job j in a schedule is the sum of the lengths of the jobs up to and including j.

The goal is to minimize the WEIGHTED SUM OF COMPLETION TIMES

$$\sum_{j=1}^{n} w_j C_j.$$

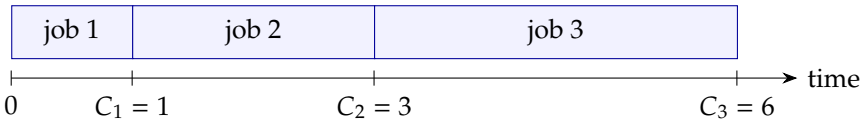


Figure 3: Three jobs with $\ell_1 = 1$, $\ell_2 = 2$, $\ell_3 = 3$, scheduled in the order $1, 2, 3$. The completion times are $C_1 = 1$, $C_2 = 3$, $C_3 = 6$.

Example: With $w_1 = 3$, $w_2 = 2$, $w_3 = 1$, the schedule in the figure has objective $3 \cdot 1 + 2 \cdot 3 + 1 \cdot 6 = 15$.

4.2 Designing the greedy algorithm

Two special cases suggest the right answer. With equal lengths, jobs with larger weight should go earlier. With equal weights, shorter jobs should go earlier. When the advice conflicts, for example $w_i > w_j$ but $\ell_i > \ell_j$, assign each job a score that increases in its weight and decreases in its length, and schedule jobs in order of decreasing score. Two natural guesses are:

1. order jobs by decreasing difference $w_j - \ell_j$;
2. order jobs by decreasing ratio w_j / ℓ_j.

To rule one out, find an instance on which the two orders differ; at least one of them is then incorrect.

Example: Take $\ell_1 = 5$, $w_1 = 3$ and $\ell_2 = 2$, $w_2 = 1$. Job 1 has the larger ratio ($3/5 > 1/2$) and job 2 the larger difference ($-1 > -2$). The difference order $2, 1$ gives $C_2 = 2$, $C_1 = 7$ and objective $1 \cdot 2 + 3 \cdot 7 = 23$. The ratio order $1, 2$ gives $C_1 = 5$, $C_2 = 7$ and objective $3 \cdot 5 + 1 \cdot 7 = 22$.

So the difference rule is not always correct. The ratio rule is.

Algorithm 4.1: Greedy scheduling by ratio

Input: Jobs $1, \dots, n$ with weights w_j and lengths $\ell_j > 0$.
Output: A schedule minimizing $\sum_j w_j C_j$.

1 **sort** the jobs by nonincreasing ratio w_j/ℓ_j, breaking ties arbitrarily
2 **return** the jobs in that order

The running time is $O(n \log n)$, since the algorithm only needs to sort.

4.3 Correctness with distinct ratios

The key computation is the effect of swapping two consecutive jobs.

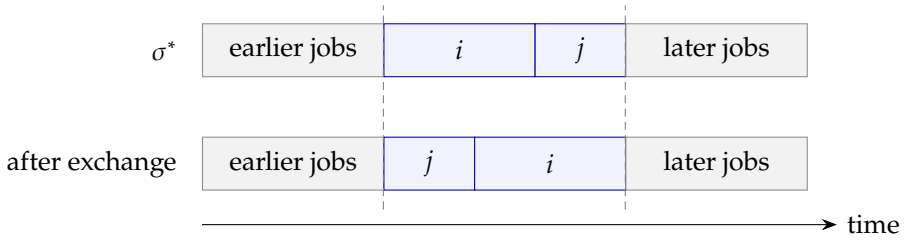


Figure 4: Exchanging consecutive jobs i and j, with i scheduled immediately before j. Jobs before and after the pair keep their completion times.

Lemma 4.2 [Cost of an exchange]. *Let i be scheduled immediately before j, and exchange them, leaving the other jobs unchanged. Then every other job's completion time is unaffected, C_i goes up by ℓ_j, and C_j goes down by ℓ_i. The objective changes by*

$$w_i \ell_j - w_j \ell_i.$$

Proof. A job before the pair is unaffected. A job after the pair still completes after the same set of jobs, including both i and j, so its completion time is unchanged. Job i now also waits for j, and job j no longer waits for i. The cost of the exchange is $w_i \ell_j$ and the benefit is $w_j \ell_i$. □

Theorem 4.3 [Greedy by ratio, distinct ratios]. *If all ratios w_j/ℓ_j are distinct, ordering the jobs by decreasing ratio minimizes $\sum_j w_j C_j$.*

Proof. The proof is by an exchange argument, by contradiction. Fix an input of n jobs and rename them so that

$$\frac{w_1}{\ell_1} > \frac{w_2}{\ell_2} > \cdots > \frac{w_n}{\ell_n}.$$

The greedy schedule σ is then $1, 2, \ldots, n$. Suppose an optimal schedule σ^* is strictly better than σ, so $\sigma^* \neq \sigma$. The only schedule in which the indices of consecutive jobs always go up is $1, 2, \ldots, n$, so σ^* has consecutive jobs i, j, with i immediately before j and $i > j$.
Exchange i and j in σ^*. Since $i > j$,

$$\frac{w_i}{\ell_i} < \frac{w_j}{\ell_j} \quad \Longrightarrow \quad w_i\ell_j < w_j\ell_i,$$

so by Lemma 4.2 the cost is less than the benefit. The exchange strictly improves σ^*, contradicting its optimality. □

4.4 Handling ties

With ties, a strictly improving exchange may not exist, so the proof shows directly that the greedy schedule is at least as good as *every* other schedule.

Theorem 4.4 [Greedy by ratio, with ties]. *Ordering the jobs by nonincreasing ratio w_j/ℓ_j, with ties broken arbitrarily, minimizes $\sum_j w_j C_j$.*

Proof. Fix an input and rename the jobs so that the greedy schedule σ is $1, 2, \ldots, n$; then $w_1/\ell_1 \geq w_2/\ell_2 \geq \cdots \geq w_n/\ell_n$. Let σ^* be an arbitrary schedule. Call a pair of jobs $i > j$ with i scheduled earlier an inverted pair.
If $\sigma^* = \sigma$, there is nothing to prove. Otherwise, as before, σ^* has consecutive jobs i, j with i immediately before j and $i > j$, an adjacent inversion. Since $i > j$,

$$\frac{w_i}{\ell_i} \leq \frac{w_j}{\ell_j} \quad \Longrightarrow \quad w_i\ell_j \leq w_j\ell_i,$$

so by Lemma 4.2 exchanging i and j has net benefit $w_j\ell_i - w_i\ell_j \geq 0$. The exchange changes the relative order of only the pair i, j, so it reduces the number of inverted pairs by exactly one. Repeat. There are at most $\binom{n}{2}$ inverted pairs, so after at most $\binom{n}{2}$ such exchanges no inversion remains, and the schedule is σ. No exchange increased the objective, so σ is at least as good as σ^*. Since σ^* was arbitrary, the greedy schedule is optimal. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), A Scheduling Application.

Lecture 5: Prim's Minimum Spanning Tree Algorithm

The informal goal is to connect a set of points as cheaply as possible. Applications include networking and clustering (Lecture 7). Two greedy algorithms solve the problem in $O(m \log n)$ time with suitable data structures, where m is the number of edges and n the number of vertices: Prim's algorithm (1957; also Dijkstra 1959 and Jarník 1930) and Kruskal's algorithm (1956, Lecture 6).

5.1 The minimum spanning tree problem

Definition 5.1 [Minimum spanning tree]. The input is an undirected graph $G = (V, E)$, in adjacency-list representation, with a cost c_e for each edge $e \in E$. Negative costs are allowed. A spanning tree is a set $T \subseteq E$ such that

1. T has no cycles, and
2. the subgraph (V, T) is connected: it contains a path between each pair of vertices.

The output is a minimum spanning tree (MST): a spanning tree minimizing the total cost $\sum_{e \in T} c_e$.

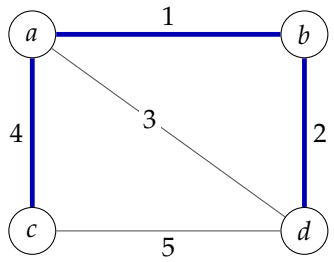


Figure 5: The running example. The highlighted edges ab, bd, ac form the minimum spanning tree, of cost 7. The set $\{ab, ad\}$ has no cycle but misses c, so it is not spanning; a set containing a cycle is disallowed.

Two standing assumptions hold throughout.

1. *G is connected.* Otherwise there is no spanning tree. This is easy to check in preprocessing, for example by depth-first search.
2. *Edge costs are distinct.* Prim's and Kruskal's algorithms remain correct with ties broken arbitrarily, but the correctness proof is more annoying, so it is omitted.

Since G is connected, $m \geq n - 1$.

5.2 Prim's algorithm

Prim's algorithm grows a single tree from an arbitrary start vertex. Each iteration increases the number of spanned vertices in the cheapest way possible, much as Dijkstra's shortest-path algorithm grows its set of processed vertices.

Algorithm 5.1: Prim's algorithm

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs c_e.
Output: A spanning tree T of G.

$X \leftarrow \{s\}$ for an arbitrary $s \in V$
$T \leftarrow \emptyset$ *// invariant: X is the vertex set spanned by T*
while $X \neq V$ **do**
  Let $e = (u, v)$ be the cheapest edge of G with $u \in X$, $v \notin X$
  $T \leftarrow T \cup \{e\}$; $X \leftarrow X \cup \{v\}$
return T

Example: In Figure 5, start at $s = a$. The edges leaving $\{a\}$ cost $1, 3, 4$, so ab is added. The edges leaving $\{a, b\}$ cost $2, 3, 4$, so bd is added. The edges leaving $\{a, b, d\}$ are ac and cd, so ac is added. The result is the minimum spanning tree $\{ab, bd, ac\}$.

Theorem 5.2 [Correctness of Prim's algorithm]. *Prim's algorithm always outputs a minimum spanning tree.*

The proof has two parts. Part I shows that the output T^* is a spanning tree, using basic facts about cuts. Part II shows that T^* is a minimum spanning tree, using the cut property. Both tools are reused for Kruskal's algorithm.

5.3 Cuts and three graph facts

Definition 5.3 [Cut]. A cut of a graph $G = (V, E)$ is a partition of V into two nonempty sets A and B. An edge crosses the cut (A, B) if it has one endpoint in each set.

A graph with n vertices has roughly 2^n cuts: each vertex chooses whether it lies in A or in B. (Exactly $2^n - 2$ ordered choices leave both sides nonempty.)

Lemma 5.4 [Empty cut lemma]. *A graph is not connected if and only if there is a cut (A, B) with no crossing edges.*

Proof. ($\Leftarrow$) Pick $u \in A$ and $v \in B$. A u–v path starts in A and ends in B, so one of its edges crosses (A, B). No edge crosses, so there is no u–v path and G is not connected.
($\Rightarrow$) Suppose G has no u–v path. Let A be the set of vertices reachable from u (its connected component) and let $B = V \setminus A$ (all other components). Both are nonempty, since $u \in A$ and

$v \in B$. No edge crosses (A, B); otherwise its endpoint in B would also be reachable from u, and A would be bigger. □

Lemma 5.5 [Double-crossing lemma]. *If a cycle $C \subseteq E$ has an edge crossing the cut (A, B), then some other edge of C also crosses (A, B).*

Proof. Traverse C starting in A at an endpoint of the crossing edge, beginning along that edge. The traversal moves to B and must return to its starting vertex in A, which requires another crossing edge of C. □

Corollary 5.6 [Lonely cut corollary]. *If e is the only edge crossing some cut (A, B), then e lies in no cycle.*

Proof. If e lay in a cycle, Lemma 5.5 would give another edge of that cycle crossing (A, B). □

5.4 Part I: Prim's algorithm outputs a spanning tree

Proposition 5.7. *Prim's algorithm outputs a spanning tree T^*.*

Proof. We check three properties.

1. *T spans X.* This holds initially, and each iteration adds one edge joining the old tree to the one new vertex, so it holds by induction.
2. *The algorithm cannot get stuck with $X \neq V$.* Otherwise no edge crosses the cut $(X, V \setminus X)$, and Lemma 5.4 says G is disconnected, contrary to the standing assumption.
3. *No cycle is ever created in T.* Consider an iteration with current sets X and T, in which e is added. Every edge of T has both endpoints in X, so e is the first edge crossing $(X, V \setminus X)$ to be added to T. It is lonely in this cut within $T \cup \{e\}$, so by Corollary 5.6 its addition cannot create a cycle.

At termination $X = V$, so T^* connects all vertices and has no cycles. □

5.5 Part II: the cut property

The key question is when it is *safe* to include an edge in the tree so far.

Theorem 5.8 [Cut property]. *Assume distinct edge costs. Let e be an edge of G, and suppose there is a cut (A, B) such that e is the cheapest edge of G crossing it. Then e belongs to every minimum spanning tree of G.*

For example, take a 4-cycle whose edges cost $1, 2, 3, 4$ in order around the cycle. The cut isolating the vertex shared by edges 1 and 2 justifies edge 1. The cut isolating the vertex shared by edges 2 and 3 justifies edge 2. The cut isolating the vertex shared by edges 3 and 4 justifies edge 3. These three edges form the minimum spanning tree.

Proof that the cut property implies Theorem 5.2. By Proposition 5.7, Prim's algorithm outputs a spanning tree T^*. Every edge it adds is the cheapest edge crossing the current cut $(X, V \setminus X)$, so each edge of T^* is justified by the cut property. Hence T^* is contained in every minimum spanning tree T. Adding an edge $e \notin T^*$ to the spanning tree T^* creates a cycle with the path in T^* joining its endpoints, so a spanning tree cannot strictly contain T^*. Thus $T = T^*$: the minimum spanning tree is unique, and Prim's algorithm finds it. □

5.6 Proof of the cut property

The proof is by contradiction, through an exchange argument similar to the scheduling application of Lecture 4. Suppose e is the cheapest edge crossing (A, B), but e is not in a minimum spanning tree T^*. The idea is to exchange e with an edge of T^* and obtain a cheaper spanning tree.

Since T^* is connected, some edge $f \neq e$ of T^* crosses (A, B), and $c_e < c_f$. Exchanging an arbitrary such f can fail, however: for a spanning tree T^* with $e \notin T^*$ and $f \in T^*$, the set $T^* \cup \{e\} \setminus \{f\}$ may or may not be a spanning tree, depending on the choice of e and f, even when e is the cheapest edge crossing a cut.

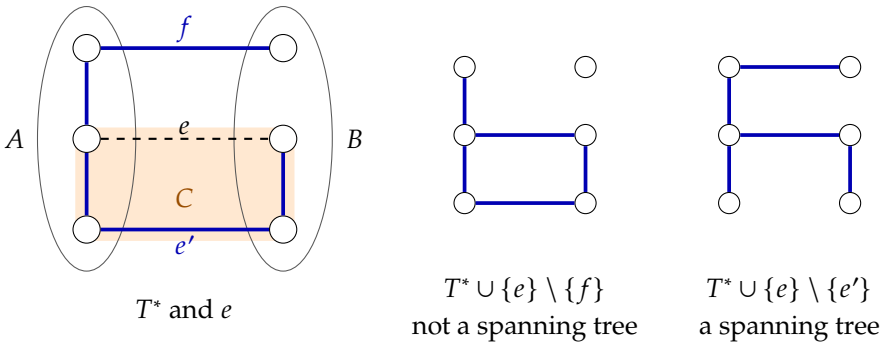


Figure 6: Left: a spanning tree T^* (solid), a non-tree edge e crossing (A, B), and the cycle C (shaded) that e creates. Both f and e' cross the cut. Middle: exchanging e with f isolates a vertex and leaves a cycle. Right: exchanging e with $e' \in C$ gives a spanning tree.

Proof of Theorem 5.8. Since T^* is a spanning tree, it contains a path between the endpoints of e; adding e closes this path into a cycle $C \subseteq T^* \cup \{e\}$. Because e crosses (A, B), Lemma 5.5 gives another edge $e' \neq e$ of C crossing (A, B), and $e' \in T^*$. Let

$$T = T^* \cup \{e\} \setminus \{e'\}.$$

T is a spanning tree. It is connected, because a path in $T^* \cup \{e\}$ that uses e' can be rerouted around the rest of C. It is acyclic, because every cycle of $T^* \cup \{e\}$ uses e and the path in T^* between its endpoints, which is unique; so C is the only cycle, and removing e' breaks it. Since e is the cheapest edge crossing the cut, $c_e < c_{e'}$, and therefore

$$\sum_{g \in T} c_g = \sum_{g \in T^*} c_g + c_e - c_{e'} < \sum_{g \in T^*} c_g.$$

This contradicts the minimality of T^*. □

5.7 A fast implementation with heaps

A straightforward implementation performs $n - 1$ iterations, each scanning all m edges for the cheapest crossing edge in $O(m)$ time, for $O(mn)$ total. As with Dijkstra's algorithm, a heap does better. Maintain:

1. the heap contains exactly the vertices of $V \setminus X$;
2. for $v \in V \setminus X$, key$[v]$ is the cost of the cheapest edge (u, v) with $u \in X$, or $+\infty$ if there is no such edge.

Computing the initial keys takes $O(m)$ time, and the $n - 1$ inserts take $O(n \log n)$, so initialization costs $O(m + n \log n) = O(m \log n)$. Given the invariants, ExtractMin returns the next vertex $v \notin X$, and the edge achieving its key is the edge (u, v) crossing $(X, V \setminus X)$ to add to T.

When v moves into X, keys may become stale. Only vertices $w \in V \setminus X$ adjacent to v can change, and for them the new edge (v, w) is a new candidate.

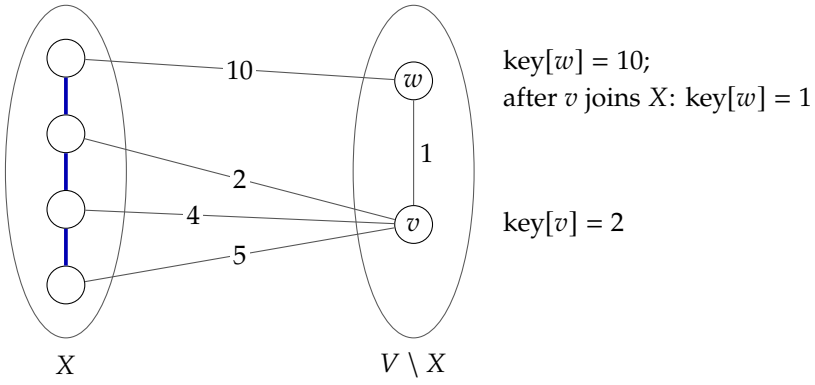


Figure 7: Maintaining the keys. Before the iteration, key$[v] = 2$ and key$[w] = 10$. Prim's algorithm extracts v and adds the edge of cost 2. The edge (v, w) now crosses the new cut, so key$[w]$ drops to 1.

Algorithm 5.2: Prim's algorithm with a heap

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs c_e.
Output: A minimum spanning tree T of G.

```
X ← {s}; T ← ∅
for each v ∈ V \ {s} do
    key[v] ← min({c_sv : (s, v) ∈ E} ∪ {+∞}); insert v into the heap
while X ≠ V do
    v ← ExtractMin; add v to X and its key-achieving edge to T
    for each edge (v, w) ∈ E with w ∈ V \ X do
        Delete w from the heap
        key[w] ← min{key[w], c_vw}
        Reinsert w into the heap
return T
```

Deleting w from the middle of the heap requires knowing where w sits, and the implementation must also remember which edge achieves each key; this bookkeeping is the subtle part of the implementation.

Theorem 5.9 [Running time with heaps]. *Prim's algorithm with a heap runs in $O(m \log n)$ time.*

Proof. The running time is dominated by the heap operations. There are $n - 1$ inserts during preprocessing and $n - 1$ extractions, one per iteration. Each edge (v, w) triggers at most one delete and reinsert, when its first endpoint is added to X. This is $O(m)$ heap operations, using $m \geq n - 1$, each taking $O(\log n)$ time, for $O(m \log n)$ total, as fast as sorting. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Prim's Minimum Spanning Tree Algorithm.

Lecture 6: Kruskal's Minimum Spanning Tree Algorithm

The problem is the same as in Lecture 5: given a connected undirected graph $G = (V, E)$ with distinct edge costs c_e, find a minimum-cost spanning tree (no cycles, connected). The main tool is again the cut property: if e is the cheapest edge crossing some cut (A, B), then e belongs to the minimum spanning tree. Throughout, $n = |V|$ and $m = |E|$.

6.1 Kruskal's algorithm

Prim's algorithm grows one tree. Kruskal's algorithm instead scans all edges from cheapest to most expensive and keeps an edge whenever it does not close a cycle, so the chosen edges may form several trees until the end.

Algorithm 6.1: Kruskal's algorithm

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs.
Output: A spanning tree T of G.

```
sort edges by increasing cost; rename them 1, 2, ..., m so that c_1 < c_2 < ... < c_m
T ← ∅
for i ← 1 to m do
    if T ∪ {i} has no cycles then
        T ← T ∪ {i}
return T
```

Example: In Figure 8, edges 1, 2, and 3 are added. Edge 4 joins two vertices already connected by edges 2 and 3, so it is skipped. Edge 5 reaches the last vertex. Edges 6 and 7 would create cycles. The minimum spanning tree $\{1, 2, 3, 5\}$ has cost 11.

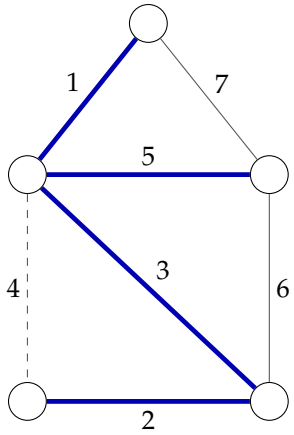


Figure 8: Kruskal's algorithm on a five-vertex graph. Edges 1, 2, 3, 5 are added (highlighted). Edge 4 (dashed) is rejected because it would close the cycle through edges 2 and 3; edges 6 and 7 are rejected for the same reason.

6.2 Correctness

The proof uses three facts from Lecture 5: the *empty cut lemma* (a graph is disconnected if and only if some cut has no crossing edges), the *lonely cut corollary* (an edge that is the only one crossing some cut lies in no cycle), and the cut property.

Theorem 6.1 [Correctness of Kruskal's algorithm]. *Kruskal's algorithm outputs the minimum spanning tree of G.*

Proof. Let T^* be the output.

1. *T^* has no cycles*, since the algorithm never adds an edge that creates one.
2. *T^* is connected.* By the empty cut lemma, it suffices to show that T^* has an edge crossing every cut (A, B). Since G is connected, at least one of its edges crosses (A, B). The key point is that Kruskal's algorithm includes the *first* edge crossing (A, B) that it considers: when that edge is examined, it is the only edge of $T \cup \{i\}$ crossing (A, B), so by the lonely cut corollary it cannot create a cycle.
3. *Every edge of T^* is justified by the cut property.* Consider the iteration in which the edge (u, v) is added to the current set T. Since $T \cup \{(u, v)\}$ has no cycle, T has no u–v path. As in the proof of the empty cut lemma, let A be the set of vertices reachable from u in (V, T) and $B = V \setminus A$; then $v \in B$ and no edge of T crosses (A, B). By the key point of (2), no edge crossing (A, B) was considered earlier: the first one considered would have been added to T, and an edge of T would cross (A, B). Hence (u, v) is the first, and therefore the cheapest, edge of G crossing (A, B), and the cut property justifies it.

By (1) and (2), T^* is a spanning tree. By (3) and the cut property, every edge of T^* lies in the minimum spanning tree; a spanning tree contained in a spanning tree equals it, so T^* is the minimum spanning tree. □

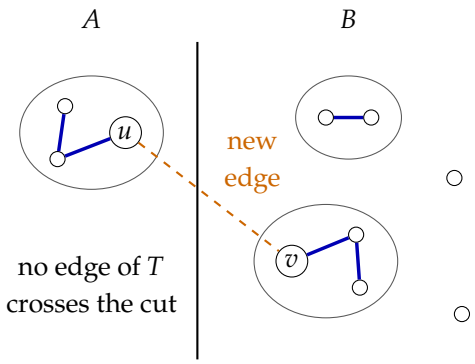


Figure 9: Step (3) of the proof. The blobs are the connected components of the current (V, T). Taking A to be the component of u gives a cut that T does not cross, and the new edge (u, v) is the first edge of G crossing it.

6.3 Running time of the straightforward implementation

- Sorting takes $O(m \log m) = O(m \log n)$ time, because $m = O(n^2)$ when there are no parallel edges, so $\log m \le 2 \log n + O(1)$.
- There are m iterations. Checking whether $T \cup \{i\}$ has a cycle amounts to checking whether the endpoints of i are already connected in (V, T), which breadth-first or depth-first search does in $O(n)$ time, since (V, T) has at most $n - 1$ edges.

The total is $O(m \log n) + O(mn) = O(mn)$. A data structure supporting $O(1)$-time cycle checks reduces this to $O(m \log n)$.

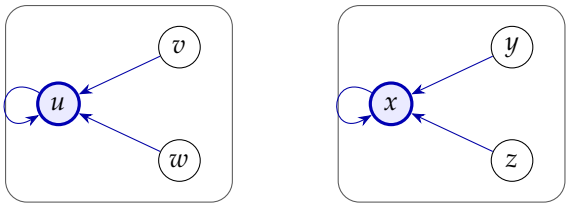
6.4 The union-find data structure

A UNION-FIND data structure maintains a partition of a set of objects into groups, supporting two operations:

- Find(x): return the name of the group containing x;
- Union(C_i, C_j): fuse the groups C_i and C_j into a single group.

For Kruskal's algorithm, the objects are the vertices and the groups are the connected components of (V, T) for the chosen edges T. Adding a new edge (u, v) to T corresponds to fusing the components of u and v, and $T \cup \{(u, v)\}$ has a cycle exactly when u and v are already in the same component.

Idea 1. Keep one linked structure per connected component of (V, T), and give each component an arbitrary LEADER vertex. Maintain the invariant that each vertex points to the leader of its component; the component's name is inherited from its leader.



leaders u and x (shaded) point to themselves

Figure 10: Two components, each with a leader. Every vertex points directly to the leader of its component, so Find is one pointer lookup.

Given an edge (u, v), the vertices u and v lie in the same component if and only if their leader pointers match, that is, Find(u) = Find(v). This is an $O(1)$-time cycle check.

When (u, v) is added to T, the components of u and v merge, and leader pointers must be updated to restore the invariant. In the worst case this takes $\Theta(n)$ updates, for example when two components

with $n/2$ vertices each merge.

Idea 2. When two components merge, the smaller one inherits the leader of the larger one. A size field in each component makes this easy. A single merge can still take $\Theta(n)$ updates, for the same reason as before. The gain appears when counting per vertex.

Lemma 6.2 [Leader updates per vertex]. *With Idea 2, each vertex has its leader pointer updated at most $\log_2 n$ times over the course of Kruskal's algorithm.*

Proof. A vertex v is updated only when its component is the smaller one in a merge, so the size of v's component at least doubles each time. Its size starts at 1 and never exceeds n, so this happens at most $\log_2 n$ times. □

Algorithm 6.2: Kruskal's algorithm with union-find

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs.
Output: The minimum spanning tree T of G.

```
sort edges by increasing cost
T ← ∅; make each vertex its own component and its own leader
for each edge (u, v) in sorted order do
    if Find(u) ≠ Find(v) then
        T ← T ∪ {(u, v)}
        Union the two components, the smaller adopting the larger one's leader
return T
```

Theorem 6.3 [Running time with union-find]. *Kruskal's algorithm with union-find runs in $O(m \log n)$ time.*

Proof. Sorting takes $O(m \log n)$ time. The m cycle checks take $O(1)$ time each, or $O(m)$ in total. By Lemma 6.2, all leader pointer updates together take $O(n \log n)$ time. Since $m \geq n - 1$, the total is $O(m \log n)$, matching Prim's algorithm. □

6.5 State of the art and open questions

Faster minimum spanning tree algorithms than $O(m \log n)$ are known:

- an $O(m)$-time randomized algorithm (Karger, Klein, Tarjan, 1995);
- an $O(m\, \alpha(n))$-time deterministic algorithm (Chazelle, 2000).

Here α is the inverse Ackermann function. It grows much more slowly than $\log^* n$, the number of times $\log_2$ can be applied to n before the result drops below 1; $\log^*$ is in turn the inverse of the tower function, whose value at k is an exponent tower of k twos.

Pettie and Ramachandran (2002) gave a deterministic minimum spanning tree algorithm that is provably optimal, yet its precise asymptotic running time is unknown: it lies between $\Theta(m)$ and $\Theta(m\,\alpha(n))$. Open questions include a *simple* randomized $O(m)$-time algorithm (it suffices to find one for the *MST verification* problem) and whether there is a deterministic $O(m)$-time algorithm. Eisner (1997) is suggested for further reading.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Kruskal's Minimum Spanning Tree Algorithm.

Lecture 7: Clustering

Clustering, also called unsupervised learning, aims to classify n "points" (web pages, images, genome fragments, and so on) into coherent groups. Kruskal's algorithm, stopped early, solves a natural formalization.

7.1 Max-spacing k-clusterings

The input is a dissimilarity measure: a distance $d(p, q)$ for each pair of points, assumed symmetric, $d(p, q) = d(q, p)$. Examples include Euclidean distance and genome similarity. The goal is that points lie in the same cluster if and only if they are nearby.

Assume the number k of desired clusters is known; in practice one can experiment with a range of values. A k-clustering partitions the points into k nonempty clusters. Points p and q are separated if they are assigned to different clusters.

Definition 7.1 [Spacing]. The spacing of a k-clustering is

$$\min_{p,q \text{ separated}} d(p, q).$$

Bigger spacing is better. The max-spacing k-clustering problem asks, given d and k, for a k-clustering of maximum spacing.

7.2 Single-link clustering

Algorithm 7.1: Greedy (single-link) clustering

Input: Points with symmetric distances $d(p, q)$; an integer $2 \le k \le n$.
Output: A k-clustering.

```
Put each point in a separate cluster
while there are more than k clusters do
    Let p, q be the closest pair of separated points        // determines the current spacing
    Merge the clusters containing p and q
return the clusters
```

This is Kruskal's algorithm stopped early, under the correspondence

$$\text{points} \leftrightarrow \text{vertices}, \qquad \text{distances} \leftrightarrow \text{edge costs}, \qquad \text{point pairs} \leftrightarrow \text{edges}.$$

A pair inside one cluster is not separated and is skipped, just as Kruskal's algorithm skips an edge that would create a cycle. The method is called single-link clustering. We say x and y were

DIRECTLY MERGED if the pair x, y was chosen in some iteration.

7.3 Correctness

Theorem 7.2 [SINGLE-LINK CLUSTERING IS OPTIMAL]. *Single-link clustering computes a max-spacing k-clustering.*

Let $C_1, \ldots, C_k$ be the greedy clustering and S its spacing, and let $\hat{C}_1, \ldots, \hat{C}_k$ be an arbitrary k-clustering. We must show that the spacing of $\hat{C}_1, \ldots, \hat{C}_k$ is at most S.

Lemma 7.3 [MERGE DISTANCES]. *If x and y were directly merged at some point, then $d(x, y) \le S$.*

Proof. Merging only removes pairs from the set of separated pairs, so the distance of the closest separated pair never decreases. The pair x, y was the closest separated pair when it was merged, and S is the distance of the closest separated pair at the end, so $d(x, y) \le S$. □

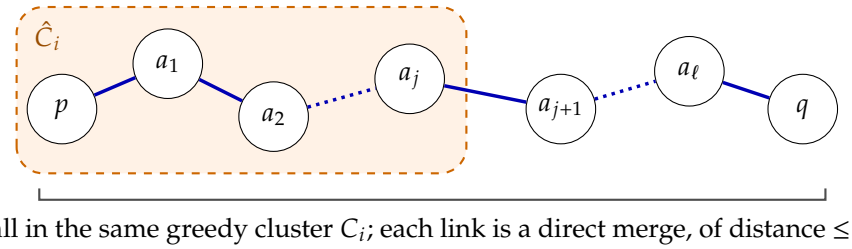


Figure 11: The tricky case. All points shown lie in one greedy cluster C_i, linked by direct merges of distance at most S. The path leaves $\hat{C}_i$ at some step $a_j \to a_{j+1}$, and that separated pair has distance at most S.

Proof of Theorem 7.2. Case 1: the $\hat{C}_i$ are the C_i, possibly after renaming. Then the spacing is the same, S.
Case 2: otherwise. Then there are points p, q in the same greedy cluster C_i but in different clusters $\hat{C}_i$ and $\hat{C}_j$. Indeed, if no such pair existed, each greedy cluster would lie inside a single $\hat{C}$-cluster; each $\hat{C}$-cluster would then be a union of greedy clusters and, being nonempty, would contain at least one, so with k clusters on each side the two clusterings would coincide, which is Case 1.
Easy case: p and q were directly merged. Since p, q are separated in $\hat{C}_1, \ldots, \hat{C}_k$, Lemma 7.3 gives

$$S \ge d(p, q) \ge \text{spacing of } \hat{C}_1, \ldots, \hat{C}_k.$$

Tricky case: p and q were merged indirectly, through several direct merges. The direct merges inside C_i connect all of its points, so there is a path $p = a_0, a_1, \ldots, a_\ell, a_{\ell+1} = q$ of points of C_i in which each consecutive pair was directly merged. Since $a_0 = p \in \hat{C}_i$ and $a_{\ell+1} = q \notin \hat{C}_i$, some

consecutive pair satisfies $a_j \in \hat{C}_i$ and $a_{j+1} \notin \hat{C}_i$. Then

$$S \;\geq\; d(a_j, a_{j+1}) \;\geq\; \text{spacing of } \hat{C}_1, \ldots, \hat{C}_k,$$

where the first inequality holds by Lemma 7.3 because a_j, a_{j+1} were directly merged, and the second because a_j, a_{j+1} are separated in $\hat{C}_1, \ldots, \hat{C}_k$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Clustering.

Lecture 8: Advanced Union-Find

A union-find data structure maintains a partition of a set X of n objects into groups and supports two operations:

- Find(x): return the name of the group containing x;
- Union(x, y): merge the groups containing x and y.

In the implementation used for Kruskal's algorithm, each object points directly to the leader of its group. Then Find takes $O(1)$ time. When two groups merge, the smaller group inherits the leader of the larger one, so each object's pointer changes only when its group at least doubles in size, and n unions take $O(n \log n)$ total work. This lecture trades slower Finds for much cheaper Unions and then shows that two simple optimizations make both operations almost constant time on average.

8.1 Lazy unions

Instead of updating every pointer in the smaller group, a lazy union updates only one: the leader (root) of one group becomes a child of the leader of the other.

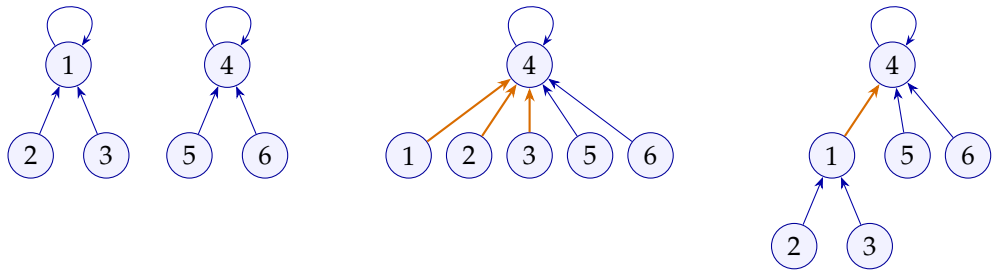


Figure 12: Merging the groups led by 1 and 4. Arrows are parent pointers; a loop marks a root. With leader pointers, objects 1, 2, 3 are all redirected to 4. A lazy union redirects only object 1; objects 2 and 3 now reach the leader in two hops.

In array form, with $A[i]$ the parent of object i:

Object i	1	2	3	4	5	6
Before the union	1	1	1	4	4	4
Leader pointers	4	4	4	4	4	4
Lazy union	4	1	1	4	4	4

Each object x now has a field parent$[x]$. The parent pointers induce a collection of directed trees on X, one per group, and x is a root if and only if parent$[x] = x$. The root serves as the group's name. Initially parent$[x] = x$ for every x, so each object is its own group.

Algorithm 8.1: Find(x) **with lazy unions**

Input: Object $x \in X$.
Output: The root of the tree containing x.

```
1 while parent[x] ≠ x do
2 |   x ← parent[x]
3 return x
```

Union(x, y) computes $s_1 =$ Find(x) and $s_2 =$ Find(y) and resets the parent of one of s_1, s_2 to be the other. So a union costs two Finds plus $O(1)$ work. The price is that recovering a leader requires following a path of parent pointers, not just one, so Find may no longer take constant time.

If the new root is chosen arbitrarily, both operations take $\Theta(n)$ time in the worst case. Repeatedly making the root of a path a child of a new singleton object lengthens the path by one each time, so $n - 1$ unions can build a single path through all n objects, and a Find from its bottom visits all n of them.

8.2 Union by rank

To avoid such long, thin trees, each object x keeps a field rank$[x]$, initially 0. For now, ranks maintain the following invariant.

Definition 8.1 [Rank]. rank$[x]$ is the maximum number of hops from a leaf of x's subtree to x. Equivalently, a leaf has rank 0 and every other object has rank $1 + \max\{\text{rank}[c] : c \text{ a child of } x\}$.

Union by rank makes the root with the smaller rank a child of the root with the larger rank. With a tie, the new root is chosen arbitrarily and its rank increases by one; otherwise no rank changes. This is exactly what restores the invariant: hanging a tree of height r_1 below a root of height $r_2 > r_1$ creates paths of length at most $r_1 + 1 \le r_2$, while with $r_1 = r_2 = r$ it creates a path of length $r + 1$.

The following properties are immediate from the invariant and the update rule.

(1) For every object x, rank$[x]$ only goes up over time.

(2) Only ranks of roots go up; once x is a non-root, its rank is frozen forever.

(3) Ranks strictly increase along every path toward the root.

The maximum rank is roughly the worst-case running time of Find, since a root of rank r has a leaf r hops below it.

Algorithm 8.2: Union(x, y) **with union by rank**

Input: Objects $x, y \in X$.

$s_1 \leftarrow \text{Find}(x)$; $s_2 \leftarrow \text{Find}(y)$
if $s_1 = s_2$ **then return**
if $\text{rank}[s_1] > \text{rank}[s_2]$ **then**
  $\text{parent}[s_2] \leftarrow s_1$
else
  $\text{parent}[s_1] \leftarrow s_2$
  if $\text{rank}[s_1] = \text{rank}[s_2]$ **then**
    $\text{rank}[s_2] \leftarrow \text{rank}[s_2] + 1$

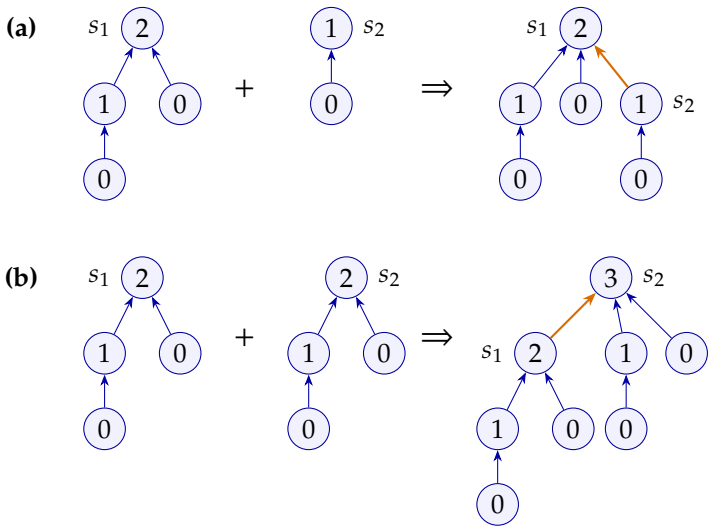


Figure 13: Union by rank; numbers inside nodes are ranks. (a) With unequal ranks, the smaller-rank root s_2 is hung below s_1 and no rank changes. (b) With equal ranks $r = 2$, the new root s_2 gets rank $r + 1 = 3$.

Theorem 8.2 [Rank Lemma]. *Consider an arbitrary sequence of* Union *and* Find *operations. At every time and for every $r \in \{0, 1, 2, \ldots\}$, at most $n/2^r$ objects have rank r.*

The subtree of an object x consists of the objects from which x can be reached by following parent pointers, including x.

Proof. The lemma follows from two claims: objects of rank r have disjoint subtrees, each with at least 2^r of the n objects.
Claim 1: if $x \neq y$ have the same rank r, their subtrees are disjoint. Suppose both subtrees contain z. Following parent pointers from z gives a unique path to its root, and both x and y lie on it. So one of x, y is an ancestor of the other, and by property (3) the ancestor has strictly larger rank, a contradiction.
Claim 2: the subtree of a rank-r object has size at least 2^r. Induct on the number of operations. Initially all ranks are 0 and all subtree sizes are 1. Subtree sizes never decrease, so there is nothing to prove unless some rank changes. That happens only in Union(x, y) with

$\text{rank}[s_1] = \text{rank}[s_2] = r$, when s_2 gets rank $r+1$. Its new subtree is the union of the old subtrees of s_1 and s_2, each of size at least 2^r by the inductive hypothesis, so it has size at least 2^{r+1}. □

Corollary 8.3. *The maximum rank is always at most* $\log_2 n$*. Hence, with union by rank,* Find *and* Union *take* $O(\log n)$ *time in the worst case.*

Proof. If some object has rank r, then $1 \le n/2^r$, so $r \le \log_2 n$. Ranks strictly increase along a path to the root, so a Find visits at most $1 + \log_2 n$ objects, and a Union is two Finds plus $O(1)$ work. □

8.3 Path compression

A Find may traverse the same leaf-to-root path many times. Path compression avoids this: after Find(x), reset the parent pointer of every object on the path from x to its root directly to the root.

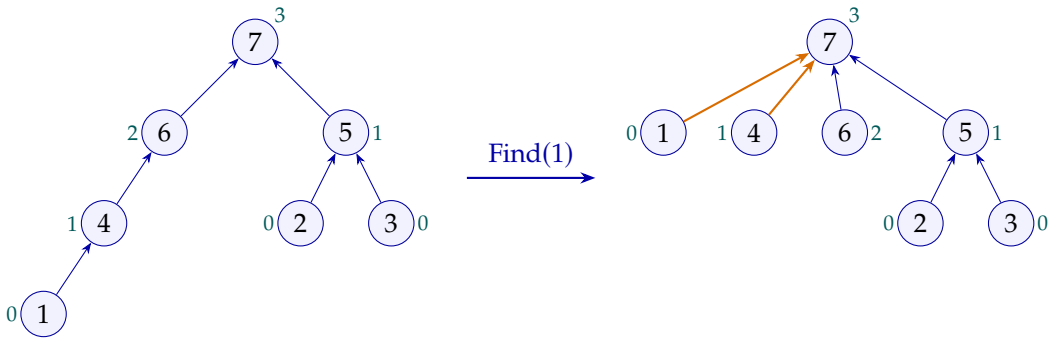


Figure 14: Find(1) with path compression. Numbers inside nodes are objects; small numbers beside them are ranks, which do not change. Objects 1 and 4 on the path now point directly to the root 7 (highlighted); 6 already did.

Algorithm 8.3: Find(x) **with path compression**

Input: Object $x \in X$.
Output: The root of the tree containing x.

```
r ← x
while parent[r] ≠ r do
    r ← parent[r]
while x ≠ r do
    q ← parent[x]; parent[x] ← r; x ← q
return r
```

The parent array before and after Find(1) in the figure:

Object i	1	2	3	4	5	6	7
parent[i] before	4	5	5	6	7	7	7
parent[i] after	7	5	5	7	7	7	7

The extra pass costs only a constant factor per Find and speeds up later ones. Ranks are maintained *exactly* as without path compression: they start at 0, the new root in a union is the old root of larger rank, and merging two roots of common rank r resets the new root's rank to $r + 1$. Compression never changes a rank.

Consequently, rank[x] is now only an upper bound on the maximum number of hops from a leaf to x, and it may be much larger. In the figure, object 6 keeps rank 2 but becomes a leaf. Two facts survive.

- The Rank Lemma still holds. Compression changes neither ranks nor which objects are roots, and each Find returns the same root as before. So every union links the same two roots and makes the same rank change as in the run without compression, ranks agree with that run at all times, and that run satisfies the lemma.
- rank[parent[x]] $>$ rank[x] for every non-root x. Compression only replaces a parent by a proper ancestor, whose rank is larger still.

8.4 The Hopcroft-Ullman bound

Definition 8.4 [Iterated logarithm]. $\log^* n$ is the number of times $\log_2$ must be applied to n before the result is at most 1.

For example, $\log^*(2^{65536}) = 5$, via $2^{65536} \mapsto 65536 \mapsto 16 \mapsto 4 \mapsto 2 \mapsto 1$. In general, if $T_1 = 2$ and $T_{t+1} = 2^{T_t}$, so that T_t is a tower of t twos, then $\log^* T_t = t$.

Theorem 8.5 [Hopcroft-Ullman 1973]. *With union by rank and path compression, m* Union *and* Find *operations take $O((m + n) \log^* n)$ time, which is $O(m \log^* n)$ in the interesting case $m = \Omega(n)$.*

Since a Union is two Finds plus $O(1)$ work, it suffices to count the objects visited by Finds. To measure progress, consider a non-root object x, whose rank is frozen, and the gap rank[parent[x]] $-$ rank[x]. Path compression increases this gap: if x's parent changes from p to $p' \neq p$, then p' is a proper ancestor of p, so rank[p'] $>$ rank[p].

Group ranks into rank blocks

$$\{0\},\ \{1\},\ \{2,3,4\},\ \{5,\ldots,2^4\},\ \{17,\ldots,2^{16}\},\ \{65537,\ldots,2^{65536}\},\ \ldots,$$

where after the first three, each block has the form $\{k+1,\ldots,2^k\}$ with k the largest rank of the

previous block. The block endpoints form a tower of twos, and ranks are at most $\log_2 n$, so only $O(\log^* n)$ blocks are needed. A step $x \to \text{parent}[x]$ makes fast progress when it moves to a larger block.

Definition 8.6 [Good and bad objects]. At a given time, an object x is good if x or its parent is a root, or if rank[parent[x]] lies in a larger rank block than rank[x]. Otherwise x is bad.

Proof of Theorem 8.5. Good objects. Along a path to the root ranks strictly increase, so the block index can increase at most once per block. Hence each Find visits at most 2 + (number of blocks) = $O(\log^* n)$ good objects: the root, its child, and the objects whose parent lies in a larger block. Over m operations this is $O(m \log^* n)$.
Bad objects. Objects of rank 0 or 1 are never bad, since a non-root's parent has larger rank and so lies in a larger block. Fix a block $\{k+1, \ldots, 2^k\}$ and an object x whose rank lies in it; while x is bad, it is a non-root, so this is its final rank. When a Find visits a bad x, its parent is not a root, so compression changes the parent to the root, which has strictly larger rank. The parent's rank starts above $k+1$, and once it exceeds 2^k, x is good forevermore. So x is visited at most 2^k times while bad. By the Rank Lemma, the number of objects with final rank in this block is at most

$$\sum_{i=k+1}^{2^k} \frac{n}{2^i} \le \frac{n}{2^k}.$$

Hence there are at most $2^k \cdot n/2^k = n$ visits to bad objects in this block. The block $\{2, 3, 4\}$ is similar: each of its fewer than $n/2$ objects is visited at most twice while bad, as its parent's rank rises from at least 3 past 4. Over all blocks there are $O(n \log^* n)$ such visits. The total work is $O((m+n) \log^* n)$. □

8.5 The Ackermann function

Many definitions of the Ackermann function exist, all more or less equivalent. Here it is defined recursively for integers $k \ge 0$ and $r \ge 1$.

Definition 8.7 [Ackermann function]. Let $A_0(r) = r + 1$. For $k \ge 1$, let $A_k(r)$ apply A_{k-1} to r, r times:

$$A_k(r) = \underbrace{(A_{k-1} \circ A_{k-1} \circ \cdots \circ A_{k-1})}_{r\text{-fold composition}}(r).$$

Unwinding the first levels:

- $A_1(r) = 2r$: add 1 to r, r times.
- $A_2(r) = r2^r$: double r, r times.

- $A_3(2) = A_2(A_2(2)) = A_2(8) = 8 \cdot 2^8 = 2^{11} = 2048$. In general, since $A_2(s) \geq 2^s$, $A_3(r)$ is at least a tower of r twos.
- $A_4(2) = A_3(A_3(2)) = A_3(2048)$, which is at least a tower of 2048 twos. In general A_4 is roughly an iterated tower function.

By induction on k, each A_k is increasing with $A_k(r) > r$, and $A_k(r) \geq A_{k-1}(r)$; these monotonicity facts are used below.

Definition 8.8 [Inverse Ackermann function]. For every $n \geq 4$, $\alpha(n)$ is the minimum k such that $A_k(2) \geq n$.

$\alpha(n)$	n	$\log^* n$	n
1	$4 \quad (A_1(2) = 4)$	1	2
2	$5, \ldots, 8 \quad (A_2(2) = 8)$	2	$3, 4$
3	$9, \ldots, 2048$	3	$5, \ldots, 16$
4	up to about a tower of 2048 twos	4	$17, \ldots, 65536$
		5	$65537, \ldots, 2^{65536}$

Thus α grows far more slowly than $\log^*$: when n is a tower of 2048 twos, $\log^* n = 2048$ while $\alpha(n) = 4$.

8.6 Tarjan's bound

Theorem 8.9 [Tarjan 1975]. *With union by rank and path compression, m* Union *and* Find *operations take $O((m+n)\alpha(n))$ time, which is $O(m\alpha(n))$ when $m = \Omega(n)$.*

The Hopcroft-Ullman analysis used two facts: the Rank Lemma, and that a compression raising x's parent from p to p' gives $\text{rank}[p'] \geq \text{rank}[p] + 1$. The new idea is a stronger version of the second: in most cases the new parent's rank is much larger, not just larger by one.

Definition 8.10 [Rank gap]. For a non-root object x with $\text{rank}[x] \geq 1$ (so its rank is fixed forever), let $\delta(x)$ be the maximum k such that

$$\text{rank}[\text{parent}[x]] \geq A_k(\text{rank}[x]).$$

Always $\delta(x) \geq 0$, because the parent's rank is at least $\text{rank}[x] + 1 = A_0(\text{rank}[x])$. Further, $\delta(x) \geq 1$ means $\text{rank}[\text{parent}[x]] \geq 2\,\text{rank}[x]$, and $\delta(x) \geq 2$ means $\text{rank}[\text{parent}[x]] \geq \text{rank}[x]\,2^{\text{rank}[x]}$. Since the parent's rank only goes up, $\delta(x)$ only goes up over time. If $\text{rank}[x] \geq 2$, then

$$\text{rank}[\text{parent}[x]] \leq \log_2 n < n \leq A_{\alpha(n)}(2) \leq A_{\alpha(n)}(\text{rank}[x]),$$

so $\delta(x) \in \{0, 1, \ldots, \alpha(n) - 1\}$.

Definition 8.11 [Good and bad objects, revisited]**.** An object x is bad if all of the following hold: x is not a root, parent$[x]$ is not a root, rank$[x] \geq 2$, and x has an ancestor y with $\delta(y) = \delta(x)$. Otherwise x is good.

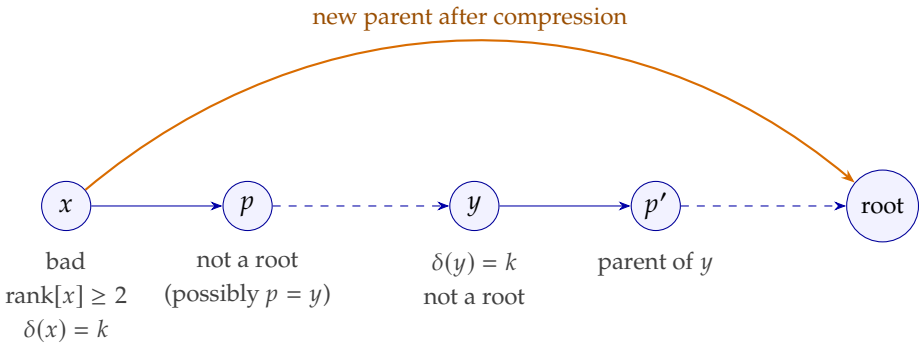


Figure 15: A Find visits a bad object x with $\delta(x) = k$. Its parent p and its ancestor y with $\delta(y) = k$ are not roots ($p = y$ is possible), and p' is the parent of y. Compression makes the root, which is p' or higher, the new parent of x.

Lemma 8.12. *Every object-to-root path contains $O(\alpha(n))$ good objects.*

Proof. Ranks strictly increase along the path, so the good objects are among: the root, the root's child, at most one object of rank 0, at most one of rank 1, and, for each $k \in \{0, \ldots, \alpha(n) - 1\}$, at most one remaining object with $\delta = k$, namely the highest one, since any lower one has that object as an ancestor with the same δ. This is at most $4 + \alpha(n)$ objects. □

Lemma 8.13. *An object x of rank $r \geq 2$ is visited at most $r\alpha(n)$ times while it is bad.*

Proof. Suppose a Find visits x while it is bad, with $\delta(x) = k$, parent p, ancestor y with $\delta(y) = k$, and p' the parent of y. After compression, x's new parent is the root, which is p' or higher, so

$$\text{rank}[\text{new parent of } x] \geq \text{rank}[p'] \geq A_k(\text{rank}[y]) \geq A_k(\text{rank}[p]).$$

The first and last inequalities hold because ranks increase toward the root and A_k is increasing; the middle one is $\delta(y) = k$. So each such visit applies (at least) A_k to the rank of x's parent. The parent's rank always exceeds r, so after r bad visits with $\delta(x) = k$ it is at least

$$\underbrace{(A_k \circ \cdots \circ A_k)}_{r \text{ times}}(r) = A_{k+1}(r)$$

by the definition of the Ackermann function, and then $\delta(x) \geq k + 1$. Thus each value of $\delta(x)$ allows at most r visits while x is bad, and $\delta(x)$ takes at most $\alpha(n)$ values. □

Proof of Theorem 8.9. By Lemma 8.12, visits to good objects cost $O(m\alpha(n))$ in total. Bad objects are non-roots, so their ranks are final, and by Lemma 8.13 and the Rank Lemma the visits to bad objects number at most

$$\sum_{\text{objects } x} \text{rank}[x]\,\alpha(n) = \alpha(n) \sum_{r \geq 0} r \cdot (\#\text{objects of rank } r) \leq n\alpha(n) \sum_{r \geq 0} \frac{r}{2^r} = 2n\alpha(n).$$

The total is $O((m+n)\alpha(n))$. □

Tarjan conjectured that no linear-time method exists and that this algorithm is optimal up to a constant factor; Fredman and Saks proved the conjecture in 1989.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Advanced Union-Find.

Lecture 9: Huffman Codes

A binary code maps each character of an alphabet Σ to a binary string. For example, if Σ consists of the letters a–z and some punctuation, 32 characters in all, the obvious encoding uses the 32 binary strings of length 5. This is a fixed-length code. When some characters are much more frequent than others, a variable-length code can do better on average.

9.1 Prefix-free codes

With variable lengths it may be unclear where one character ends and the next begins. For $\Sigma = \{A, B, C, D\}$ with encoding $\{0, 01, 10, 1\}$, the string 001 could encode AB or AAD, so it cannot be decoded without more information. The fixed-length encoding $\{00, 01, 10, 11\}$ has no such problem.

Definition 9.1 [Prefix-free code]. A code f on Σ is prefix-free if for every pair of distinct characters $i, j \in \Sigma$, neither $f(i)$ nor $f(j)$ is a prefix of the other.

For example, $\{0, 10, 110, 111\}$ is prefix-free. Such codes can give shorter encodings when character frequencies are non-uniform:

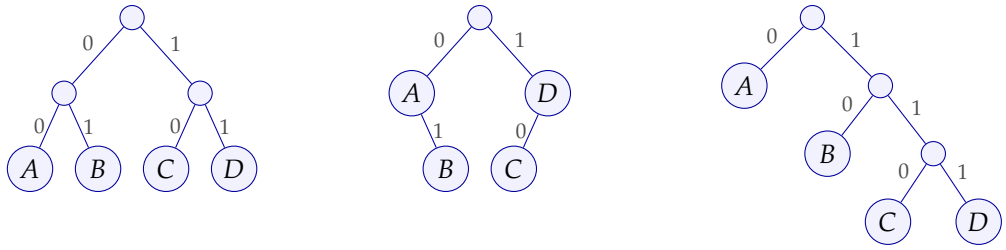
Character	Frequency	Fixed-length	Prefix-free
A	60%	00	0
B	25%	01	10
C	10%	10	110
D	5%	11	111

The fixed-length code uses 2 bits per character, while the prefix-free code uses on average

$$0.6 \cdot 1 + 0.25 \cdot 2 + (0.1 + 0.05) \cdot 3 = 1.55 \text{ bits per character.}$$

9.2 Codes as trees

Binary codes correspond to binary trees. Left child edges are labeled 0 and right child edges 1; each character $i \in \Sigma$ labels exactly one node, and the encoding of i is the sequence of bits on the path from the root to the node labeled i.



$\{00, 01, 10, 11\}$ $\{0, 01, 10, 1\}$ $\{0, 10, 110, 111\}$

Figure 16: The trees of three codes for $\Sigma = \{A, B, C, D\}$. In the middle tree, labeled nodes A and D are internal, which is why that code is not prefix-free.

One code word is a prefix of another exactly when one labeled node is an ancestor of the other. So a code is prefix-free if and only if the labeled nodes are exactly the leaves. To decode, repeatedly follow the path from the root, according to the bits, until a leaf is reached; output its label and restart at the root. Since only leaves are labeled, this is unambiguous. With $\{0, 10, 110, 111\}$, for example, 0110111 decodes as $0 \mid 110 \mid 111$, that is, ACD. The encoding length of i is the depth of i in the tree.

Definition 9.2 [Optimal prefix-free code]. The input is a probability p_i for each character $i \in \Sigma$. For a binary tree T whose leaves correspond to the symbols of Σ, the average encoding length is

$$L(T) = \sum_{i \in \Sigma} p_i \cdot [\text{depth of } i \text{ in } T].$$

The output is a binary tree T minimizing $L(T)$.

With $p_A = 0.6$, $p_B = 0.25$, $p_C = 0.1$, $p_D = 0.05$, the balanced tree of the fixed-length code has $L = 2$, while the tree of $\{0, 10, 110, 111\}$ has $L = 1.55$.

9.3 Huffman's greedy algorithm

A natural but suboptimal idea is top-down divide and conquer: partition Σ into Σ_1, Σ_2, each with about half of the total frequency, recursively build trees T_1, T_2, and make them the two subtrees of a new root.

Huffman's optimal idea builds the tree bottom-up by successive mergers. Each merger joins two current subtrees under a new root, and increases the encoding length of every symbol in them by one. So the final encoding length of $i \in \Sigma$ is the number of mergers its subtree endures. Symbols that should be deep are those of small frequency, which suggests the greedy rule: first merge the two symbols with the smallest frequencies.

To recurse after merging a and b, replace them by a new meta-symbol ab. Since ab is a proxy for "a or b," its frequency is $p_{ab} = p_a + p_b$.

Algorithm 9.1: Huffman's algorithm

Input: Alphabet Σ with $|\Sigma| \geq 2$ and frequencies p_i.
Output: A binary tree with leaves corresponding to Σ.

if $|\Sigma| = 2$ **then**
  return a root whose two children are the leaves of Σ
Let $a, b \in \Sigma$ have the smallest frequencies
$\Sigma' \leftarrow \Sigma$ with a, b replaced by a new symbol ab
$p_{ab} \leftarrow p_a + p_b$
$T' \leftarrow$ Huffman's algorithm on Σ' with frequencies p
$T \leftarrow T'$ with leaf ab split into an internal node with leaf children a and b
return T

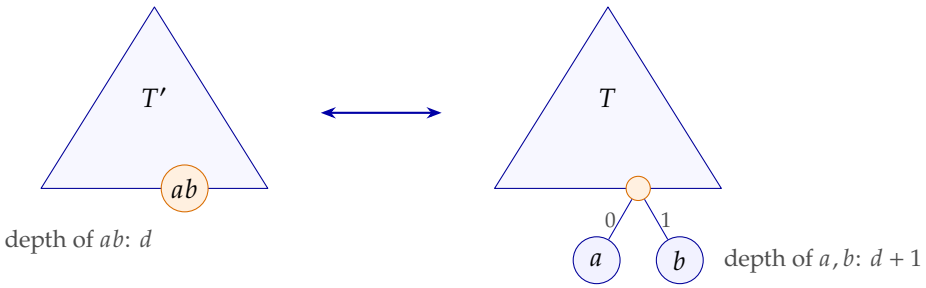


Figure 17: Extending T', whose leaves correspond to Σ', to T, whose leaves correspond to Σ, by splitting the leaf ab.

Example: For $p_A = 60$, $p_B = 25$, $p_C = 10$, $p_D = 5$ (in percent), the algorithm merges C and D into CD with frequency 15, then B and CD into BCD with frequency 40, and the base case pairs A with BCD. Splitting the meta-symbols back gives the tree of $\{0, 10, 110, 111\}$.

Example: Take characters A, B, C, D, E, F with weights $3, 2, 6, 8, 2, 6$. The algorithm merges B and E into BE (weight 4), then A and BE into ABE (weight 7), then C and F into CF (weight 12), then ABE and D into $ABDE$ (weight 15), and the base case pairs $ABDE$ with CF. The resulting code is $A = 000$, $B = 0010$, $E = 0011$, $D = 01$, $C = 10$, $F = 11$.

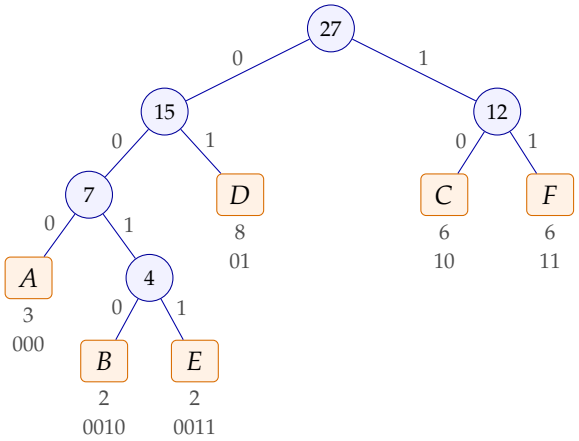


Figure 18: The final tree for the weights $3, 2, 6, 8, 2, 6$. Each internal node shows the total weight of the meta-symbol it represents; below each leaf are its weight and code word.

9.4 Correctness

Theorem 9.3 [Huffman 1952]. *Huffman's algorithm computes a binary tree, with leaves corresponding to the symbols of Σ, that minimizes the average encoding length $L(T) = \sum_{i \in \Sigma} p_i$[depth of leaf i in T].*

The proof is by induction on $n = |\Sigma| \geq 2$. When $n = 2$, the algorithm uses one bit per symbol, which is optimal since in any tree with at least two leaves every leaf has depth at least 1. For $n > 2$, let a, b be the symbols of smallest frequency, and let Σ' and $p_{ab} = p_a + p_b$ be as in the algorithm. By the inductive hypothesis, the recursive call returns a tree $\hat{T}'$ minimizing L over trees for Σ'.

Let X_{ab} be the set of trees for Σ in which a and b are siblings. Splitting the leaf ab is a bijection from trees T' for Σ' to X_{ab}. If ab has depth d in T', then a and b have depth $d + 1$ in the corresponding T, and all other depths agree, so after cancellation

$$L(T) - L(T') = p_a(d+1) + p_b(d+1) - (p_a + p_b)d = p_a + p_b.$$

This difference is independent of T and T'. Hence the tree $\hat{T}$ corresponding to $\hat{T}'$, which is the algorithm's output, minimizes L over X_{ab}. The theorem therefore follows from the next lemma: merging a and b first is safe.

Lemma 9.4 [Key lemma]. *Some tree minimizing L for Σ lies in X_{ab}.*

The intuition is that an optimal tree can only improve by pushing the least frequent symbols a and b as deep as possible.

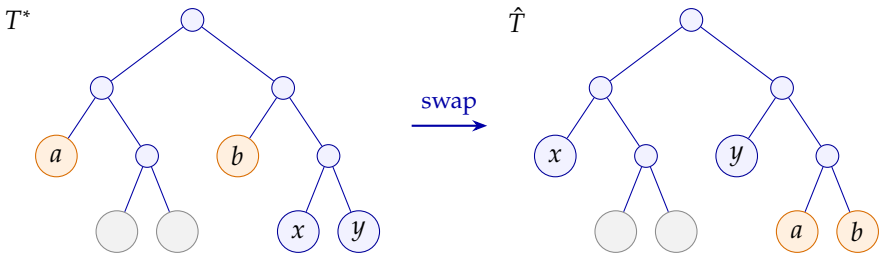


Figure 19: The exchange in the key lemma (schematic). Swapping a with x and b with y moves the two least frequent symbols to the deepest pair of sibling leaves.

Proof. Use an exchange argument. Let T^* minimize L for Σ. If an internal node has only one child, contracting it decreases the depths below it and does not increase L; so we may assume every internal node of T^* has two children. Then a deepest leaf has a sibling, which is also a leaf, since anything below it would be deeper. Let x, y be these two sibling leaves at the deepest level, named so that $x = a$ if a is one of them and $y = b$ if b is one of them.
Obtain $\hat{T}$ from T^* by swapping the labels $a \leftrightarrow x$ and $b \leftrightarrow y$. Then a and b are siblings, so

$\hat{T} \in X_{ab}$. Writing d_i for the depth of i in T^*, only the four swapped symbols change depth, and

$$L(T^*) - L(\hat{T}) = (p_x - p_a)(d_x - d_a) + (p_y - p_b)(d_y - d_b) \geq 0.$$

The frequency differences are nonnegative because a and b have the smallest frequencies, and the depth differences are nonnegative because x and y are at the deepest level. So $L(\hat{T}) \leq L(T^*)$, and $\hat{T}$ is also optimal. □

9.5 Running time

A naive implementation takes $O(n^2)$ time for $n = |\Sigma|$: there are $n-2$ recursive calls, each finding the two smallest frequencies in $O(n)$ time. A heap speeds up these repeated minimum computations. Use the frequencies as keys; after extracting the two smallest-frequency (meta-)symbols, insert the new meta-symbol with key equal to the sum of the two old keys. This gives an iterative $O(n \log n)$ implementation. Even faster, after sorting the frequencies, the remaining work can be done in $O(n)$ time by managing the (meta-)symbols with two queues.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Huffman Codes.

Lecture 10: Introduction to Dynamic Programming

Dynamic programming solves a problem by reasoning about the structure of an optimal solution, identifying a small collection of subproblems, and solving them from smallest to largest. A simple graph problem shows why greedy and divide-and-conquer approaches can fail and how the technique arises.

10.1 Weighted independent sets in path graphs

A path graph has vertices $v_1, \ldots, v_n$ and edges (v_i, v_{i+1}) for $1 \le i < n$. Each vertex v_i carries a nonnegative weight w_i.

Definition 10.1 [Independent set]. An independent set (IS) of a graph is a subset of its vertices, no two of which are adjacent. The weighted independent set problem asks for an independent set of maximum total weight.

The empty set is always independent. Brute-force search over all subsets requires exponential time.

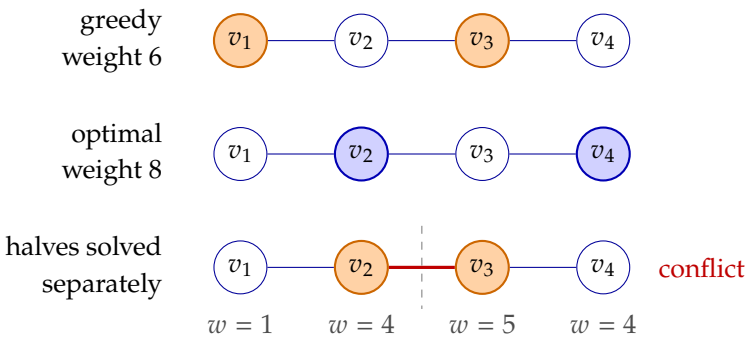


Figure 20: The path with weights $1, 4, 5, 4$. Greedy selection picks v_3, then v_1, for total weight 6; the optimum $\{v_2, v_4\}$ has weight 8. Solving each half separately picks the adjacent vertices v_2 and v_3.

The natural greedy algorithm repeatedly chooses the maximum-weight vertex not adjacent to any previously chosen vertex. On the path in Figure 20, it chooses v_3 and then v_1, for weight 6, while the maximum weight is 8.

A divide-and-conquer approach would compute a maximum-weight independent set of each half recursively and then combine them. The two recursive solutions may conflict across the boundary: in Figure 20, the halves contribute v_2 and v_3, which are adjacent. It is not clear how to repair such a conflict quickly.

10.2 Optimal substructure

The critical step is to reason about the structure of an optimal solution in terms of optimal solutions of smaller subproblems. This narrows the candidates for an optimal solution to a small set that can be searched by brute force.

Let $S \subseteq V$ be a maximum-weight independent set of G. Let G' be G with v_n deleted, and let G'' be G with v_{n-1} and v_n deleted. For $n = 1$, G'' is the empty graph.

Lemma 10.2 [Optimal substructure]. *If $v_n \notin S$, then S is a maximum-weight IS of G'. If $v_n \in S$, then $S - \{v_n\}$ is a maximum-weight IS of G''.*

Proof. *Case $v_n \notin S$.* Then S is an IS of G'. If some IS S^* of G' had larger weight, then S^* would also be an IS of G with larger weight than S, a contradiction.

Case $v_n \in S$. By independence, $v_{n-1} \notin S$, so $S - \{v_n\}$ is an IS of G''. If some IS S^* of G'' had larger weight, then $S^* \cup \{v_n\}$ would be independent in G, because S^* contains neither v_{n-1} nor v_n, and its weight would exceed that of S, a contradiction. □

Hence a maximum-weight IS of G is either a maximum-weight IS of G', or v_n together with a maximum-weight IS of G''. If we knew which case held, one recursive call would finish the job. Since we do not know, we try both possibilities and return the better solution.

10.3 Eliminating redundant recursion

The resulting recursive algorithm computes a maximum-weight IS S_1 of G' and S_2 of G'', then returns the heavier of S_1 and $S_2 \cup \{v_n\}$. It is correct by induction on n using Lemma 10.2. However, its running time satisfies $T(n) = T(n-1) + T(n-2) + O(1)$, which grows exponentially.

The recursion only removes vertices from the right end of the path, so every subproblem is a prefix of the graph. There are only $\Theta(n)$ distinct subproblems, which the exponential recursion solves over and over. One fix is memoization: the first time a subproblem is solved, store its solution in a global table for $O(1)$-time lookup later. Better still, solve the subproblems bottom-up.

Let G_i be the path on the first i vertices, with G_0 empty. Fill an array A from left to right so that $A[i]$ is the value of a maximum-weight IS of G_i.

Algorithm 10.1: Weighted independent set in a path graph

Input: Path graph with vertex weights $w_1, \dots, w_n \geq 0$.
Output: Array A with $A[i]$ the maximum weight of an IS of G_i.

```
A[0] ← 0; A[1] ← w_1
for i ← 2 to n do
    A[i] ← max{A[i − 1], A[i − 2] + w_i}
return A
```

The first argument of the maximum is case 1 (a maximum-weight IS of G_{i-1}); the second is case 2 (a maximum-weight IS of G_{i-2} plus v_i). Each iteration takes constant time, so the algorithm runs in $O(n)$ time.

Proposition 10.3 [Correctness]. *For every $0 \le i \le n$, $A[i]$ is the maximum weight of an independent set of G_i.*

Proof. Induct on i. The cases $i = 0, 1$ hold because $\emptyset$ and $\{v_1\}$ are optimal for G_0 and G_1 (weights are nonnegative). For $i \ge 2$, Lemma 10.2 applied to G_i says that an optimal IS of G_i has weight $A[i-1]$ or $A[i-2] + w_i$, and both values are attained by independent sets of G_i. Taking the maximum gives the optimum. □

10.4 Reconstructing an optimal solution

The algorithm computes the value of a maximum-weight IS, not the set. Storing an optimal set for each G_i alongside its value is correct but wasteful. Instead, trace back through the filled-in array. By the correctness of the algorithm, v_i belongs to a maximum-weight IS of G_i if and only if

$$w_i + A[i-2] \ge A[i-1].$$

For the test at $i = 1$, interpret $A[-1] = 0$.

Algorithm 10.2: Reconstruction

Input: Filled array $A[0:n]$ and weights $w_1, \dots, w_n$.
Output: A maximum-weight IS S of G.

```
S ← ∅; i ← n
while i ≥ 1 do
    if A[i − 1] ≥ A[i − 2] + w_i then          // case 1 wins
        i ← i − 1
    else                                       // case 2 wins
        S ← S ∪ {v_i}; i ← i − 2
return S
```

Proposition 10.4. *The reconstruction returns a maximum-weight IS of G in $O(n)$ time.*

Proof. Show by induction on i that the remaining loop, started at i, adds an IS of G_i of weight $A[i]$. If case 1 wins, then $A[i] = A[i-1]$ and the loop continues on G_{i-1}. Otherwise $A[i] = A[i-2] + w_i$; the loop adds v_i and continues on G_{i-2}, which does not contain the neighbor v_{i-1}, so independence is preserved. Each iteration decreases i, so there are at most n iterations. □

10.5 Principles of dynamic programming

The weighted independent set algorithm is a dynamic programming algorithm. Its key ingredients are the following.

1. Identify a small number of subproblems, such as the maximum-weight IS of G_i for $i = 0, 1, \ldots, n$.
2. Quickly and correctly solve larger subproblems given the solutions to smaller ones, usually via a recurrence such as $A[i] = \max\{A[i-1], A[i-2] + w_i\}$.
3. After solving all subproblems, quickly compute the final solution, usually the answer to the largest subproblem.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Introduction to Dynamic Programming.

Lecture 11: The Knapsack Problem

The knapsack problem chooses a subset of items of maximum total value subject to a capacity constraint. The dynamic programming algorithm follows the same three steps as for weighted independent sets: formulate a recurrence from the structure of an optimal solution, identify the subproblems, and solve them systematically.

Definition 11.1 [Knapsack problem]. The input is n items, where item i has a nonnegative value v_i and a nonnegative integral size w_i, together with a nonnegative integer capacity W. The output is a subset $S \subseteq \{1, 2, \ldots, n\}$ that maximizes $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq W$.

11.1 Optimal substructure

Let S be a maximum-value solution to a knapsack instance, and consider the last item n.

Lemma 11.2 [Optimal substructure]. *If $n \notin S$, then S is an optimal solution for the first $n-1$ items with the same capacity W. If $n \in S$, then $S - \{n\}$ is an optimal solution for the first $n-1$ items with capacity $W - w_n$.*

Proof. Case $n \notin S$. If some S^* using only the first $n-1$ items were feasible for capacity W with larger value than S, it would be equally feasible and better with respect to all n items, a contradiction.

Case $n \in S$. The set $S - \{n\}$ has total size at most $W - w_n$, so it is feasible for the smaller capacity. If some S^* using the first $n-1$ items had total size at most $W - w_n$ and higher value than $S - \{n\}$, then $S^* \cup \{n\}$ would have size at most W and value greater than S, a contradiction. □

The residual capacity in the second case is $W - w_n$, the capacity left after reserving room for item n; it is not $W - v_n$.

11.2 The recurrence and the subproblems

Let $V_{i,x}$ be the value of the best solution that uses only the first i items and has total size at most x. Applying Lemma 11.2 to this smaller instance gives, for $i \in \{1, \ldots, n\}$ and every x,

$$V_{i,x} = \max\{V_{i-1,x},\ v_i + V_{i-1,x-w_i}\}.$$

The first term is case 1 (item i excluded) and the second is case 2 (item i included). If $w_i > x$, item i cannot fit, so $V_{i,x} = V_{i-1,x}$.

The subproblems are indexed by all prefixes $\{1, 2, \ldots, i\}$ of the items, $0 \le i \le n$, and all residual capacities $x \in \{0, 1, \ldots, W\}$. Because W and the sizes w_i are integers, every residual capacity reached by the recurrence is one of these $W + 1$ values.

Algorithm 11.1: Knapsack

Input: Values $v_1, \ldots, v_n$, integral sizes $w_1, \ldots, w_n$, and capacity W.
Output: The maximum total value of a feasible subset.

$A \leftarrow$ a 2-D array indexed by $0 \le i \le n$ and $0 \le x \le W$
for $x \leftarrow 0$ **to** W **do**
  $A[0, x] \leftarrow 0$
for $i \leftarrow 1$ **to** n **do**
  for $x \leftarrow 0$ **to** W **do**
    if $w_i > x$ **then**
      $A[i, x] \leftarrow A[i-1, x]$
    else
      $A[i, x] \leftarrow \max\{A[i-1, x],\ A[i-1, x - w_i] + v_i\}$
return $A[n, W]$

Both entries on the right-hand side were computed in the previous iteration of the outer loop, so each is available for $O(1)$-time lookup.

Proposition 11.3 [Correctness and running time]. *The algorithm returns the optimal value $V_{n,W}$ in $\Theta(nW)$ time.*

Proof. By induction on i, $A[i, x] = V_{i,x}$ for every x: the case $i = 0$ holds since no items give value 0, and the inductive step is exactly the recurrence justified by Lemma 11.2. There are $(n + 1)(W + 1) = \Theta(nW)$ subproblems, each solved in $\Theta(1)$ time. □

11.3 An example

Take $n = 4$ items and capacity $W = 6$, with

$$(v_1, w_1) = (3, 4), \quad (v_2, w_2) = (2, 3), \quad (v_3, w_3) = (4, 2), \quad (v_4, w_4) = (4, 3).$$

Filling the table column by column gives Figure 21. The optimal value is $A[4, 6] = 8$.

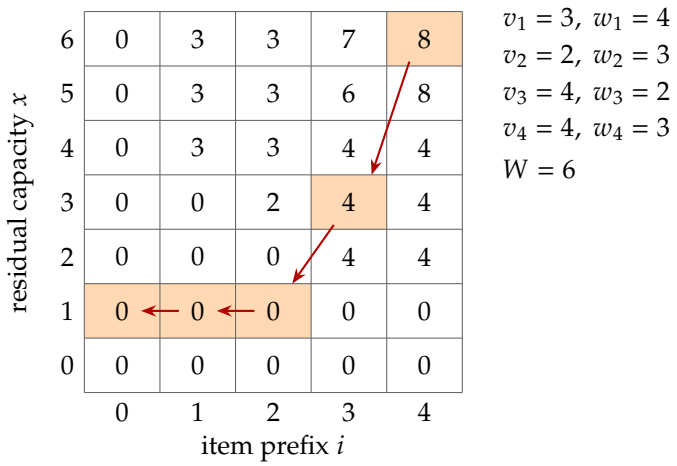


Figure 21: The filled table $A[i, x]$ for the example, with column i and row x. Shaded cells and arrows trace back an optimal solution from $A[4, 6]$.

An optimal subset can be read off by tracing back from $A[n, W]$. At entry $A[i, x]$, if $A[i, x] = A[i-1, x]$ then item i can be excluded and we move to $A[i-1, x]$; otherwise item i is included and we move to $A[i-1, x-w_i]$. Here $A[4, 6] = 8 \neq A[3, 6] = 7$, so item 4 is included and we move to $A[3, 3]$. Then $A[3, 3] = 4 \neq A[2, 3] = 2$, so item 3 is included and we move to $A[2, 1]$. Finally $A[2, 1] = A[1, 1] = A[0, 1] = 0$, so items 2 and 1 are excluded. The optimal solution is $\{3, 4\}$, with total size $5 \leq 6$ and value 8.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), The Knapsack Problem.

Lecture 12: Sequence Alignment

Sequence alignment measures the similarity of two strings by the Needleman–Wunsch score: the minimum total penalty of an alignment. Dynamic programming computes it by peeling letters off the right ends of the two strings.

12.1 Problem definition

The input is two strings $X = x_1 \cdots x_m$ and $Y = y_1 \cdots y_n$ over an alphabet Σ, such as $\{A, C, G, T\}$, together with a penalty $\alpha_{\text{gap}} \geq 0$ for each inserted gap and a penalty $\alpha_{ab} \geq 0$ for matching letters a and b, presumably with $\alpha_{ab} = 0$ when $a = b$.

Definition 12.1 [Alignment]. An alignment of X and Y inserts gaps into the two strings so that they have equal length, then pairs them position by position. Its penalty is α_{gap} for each gap plus α_{ab} for each position that pairs a letter a of X with a letter b of Y. The goal is an alignment of minimum total penalty.

Example: The alignment

A	G	G	G	C	T
A	G	G	–	C	A

of AGGGCT and AGGCA has total penalty $\alpha_{\text{gap}} + \alpha_{\text{AT}}$.

12.2 Optimal substructure

To identify subproblems, develop a recurrence from the structure of an optimal solution and then read off the subproblems it uses. Consider an optimal alignment of X and Y and its final position. There are three relevant possibilities for that position:

1. x_m and y_n are matched;
2. x_m is matched with a gap;
3. y_n is matched with a gap.

A position with two gaps is pointless: deleting it gives an alignment with no larger penalty.

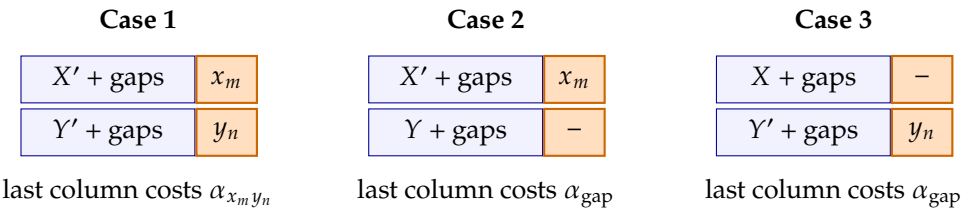


Figure 22: The three possible final positions of an optimal alignment. The remaining positions form an alignment of the indicated smaller strings.

Let $X' = x_1 \cdots x_{m-1}$ and $Y' = y_1 \cdots y_{n-1}$ denote the strings with their last letters removed.

Lemma 12.2 [Optimal substructure]. *In an optimal alignment of X and Y, the alignment induced by the other positions is optimal for X' and Y' in case 1, for X' and Y in case 2, and for X and Y' in case 3.*

Proof. Consider case 1; the other cases are similar. Suppose the induced alignment of X' and Y' has penalty P, while some other alignment of X' and Y' has penalty $P^* < P$. The original alignment has penalty $P + \alpha_{x_m y_n}$. Appending the final position x_m over y_n to the better alignment gives an alignment of X and Y with penalty

$$P^* + \alpha_{x_m y_n} < P + \alpha_{x_m y_n},$$

contradicting the optimality of the original alignment. In cases 2 and 3, the appended final position contributes α_{gap} instead. □

Thus an optimal alignment is one of only three candidates, each built from an optimal alignment of a smaller pair of strings.

12.3 Subproblems and recurrence

Since letters are only removed from the right ends, the relevant subproblems are pairs (X_i, Y_j), where X_i is the first i letters of X and Y_j is the first j letters of Y. Let P_{ij} be the penalty of an optimal alignment of X_i and Y_j. For all $i = 1, \ldots, m$ and $j = 1, \ldots, n$,

$$P_{ij} = \min \begin{cases} \alpha_{x_i y_j} + P_{i-1,j-1} & \text{case 1,} \\ \alpha_{\text{gap}} + P_{i-1,j} & \text{case 2,} \\ \alpha_{\text{gap}} + P_{i,j-1} & \text{case 3.} \end{cases}$$

This is correct because the optimal solution is one of these three candidates by Lemma 12.2, and the recurrence selects the best of them.

The base cases are $P_{i,0} = P_{0,i} = i \cdot \alpha_{\text{gap}}$: the only way to align i letters with the empty string, without pointless double-gap positions, is to match each letter with a gap.

12.4 The algorithm

Algorithm 12.1: Sequence alignment

Input: Strings $X = x_1 \cdots x_m$, $Y = y_1 \cdots y_n$, and penalties α_{gap}, α_{ab}.
Output: The minimum penalty P_{mn} of an alignment.

$A \leftarrow$ a 2-D array indexed by $0 \le i \le m$ and $0 \le j \le n$
for $i \leftarrow 0$ **to** m **do** $A[i, 0] \leftarrow i \cdot \alpha_{\text{gap}}$
for $j \leftarrow 0$ **to** n **do** $A[0, j] \leftarrow j \cdot \alpha_{\text{gap}}$
for $i \leftarrow 1$ **to** m **do**
  for $j \leftarrow 1$ **to** n **do**
    $A[i, j] \leftarrow \min\{A[i-1, j-1] + \alpha_{x_i y_j},\ A[i-1, j] + \alpha_{\text{gap}},\ A[i, j-1] + \alpha_{\text{gap}}\}$
return $A[m, n]$

All three entries on the right-hand side were filled earlier in the loop order, so each is available for $O(1)$-time lookup. By induction and the correctness of the recurrence, $A[i, j] = P_{ij}$ for all $i, j \ge 0$. The running time is $O(mn)$: $\Theta(1)$ work for each of $\Theta(mn)$ subproblems.

12.5 Reconstructing an optimal alignment

Trace back through the filled-in table, starting at $A[m, n]$. At entry $A[i, j]$:

- if $A[i, j]$ was filled using case 1, match x_i with y_j and go to $A[i-1, j-1]$;
- if it was filled using case 2, match x_i with a gap and go to $A[i-1, j]$;
- if it was filled using case 3, match y_j with a gap and go to $A[i, j-1]$.

When $i = 0$ or $j = 0$, match the remaining letters of the other string with gaps. The positions are produced from right to left. Each step decreases $i + j$, so the traceback takes only $O(m + n)$ time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Sequence Alignment.

Lecture 13: Optimal Binary Search Trees

A set of keys has many valid binary search trees. When some keys are searched for much more often than others, the best tree is not necessarily balanced, and dynamic programming finds a tree of minimum average search time.

13.1 Search trees and non-uniform access

A binary search tree satisfies the search tree property: for every node with key x, all keys in its left subtree are less than x and all keys in its right subtree are greater than x. For keys $x < y < z$, for example, the tree rooted at y with children x and z, the path $x \to y \to z$ of right children, and the path $z \to y \to x$ of left children are all valid.

A balanced search tree, such as a red-black tree, has worst-case search time $\Theta(\text{height}) = \Theta(\log n)$. With known search frequencies, however, a different tree can be better on average.

Example: Suppose 80% of searches are for x and 10% each are for y and z. Counting the number of nodes examined, the balanced tree has average search time 1.9, while the path rooted at x has average search time 1.3 (Figure 23).

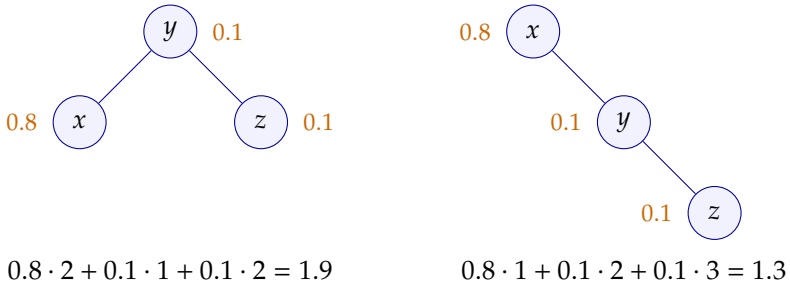


Figure 23: Average search times for keys $x < y < z$ with search frequencies 0.8, 0.1, 0.1, shown beside each node. A search for a key at depth d examines $d + 1$ nodes.

13.2 Problem definition

Definition 13.1 [Optimal binary search tree]. The input is frequencies $p_1, \ldots, p_n$ for items $1, 2, \ldots, n$, where the items are in sorted order $1 < 2 < \cdots < n$. The goal is a valid search tree T that minimizes the weighted search time

$$C(T) = \sum_{i=1}^{n} p_i \cdot [\text{search time for } i \text{ in } T],$$

where the search time for i is its depth in T plus one.

If $\sum_i p_i = 1$, then $C(T)$ is an average search time, and a red-black tree T has $C(T) = O(\log n)$.

The problem resembles Huffman coding: both output a binary tree and essentially minimize average depth with respect to given probabilities. The constraints differ. Huffman codes must be prefix-free, so symbols appear only at leaves; here every node holds a key and the tree must satisfy the search tree property, which seems harder to handle.

13.3 Greedy does not work

Intuitively, the most frequently accessed items should be closest to the root and the least frequently accessed ones furthest from it. This suggests two greedy algorithms:

- bottom-up: populate the lowest level with the least frequently accessed keys;
- top-down: put the most frequently accessed item at the root, then recurse on the two sides.

Neither is optimal in general (Figure 24). In the first instance, the greedy tree keeps the rarely accessed keys 1 and 4 on the lowest levels but forces key 3 (frequency 73) below the root, for cost 179 instead of 129. In the second instance, top-down greedy puts key 2 (frequency 34) at the root and then key 3 (frequency 33) below it, for cost 168; rooting the tree at key 3 instead gives cost 148.

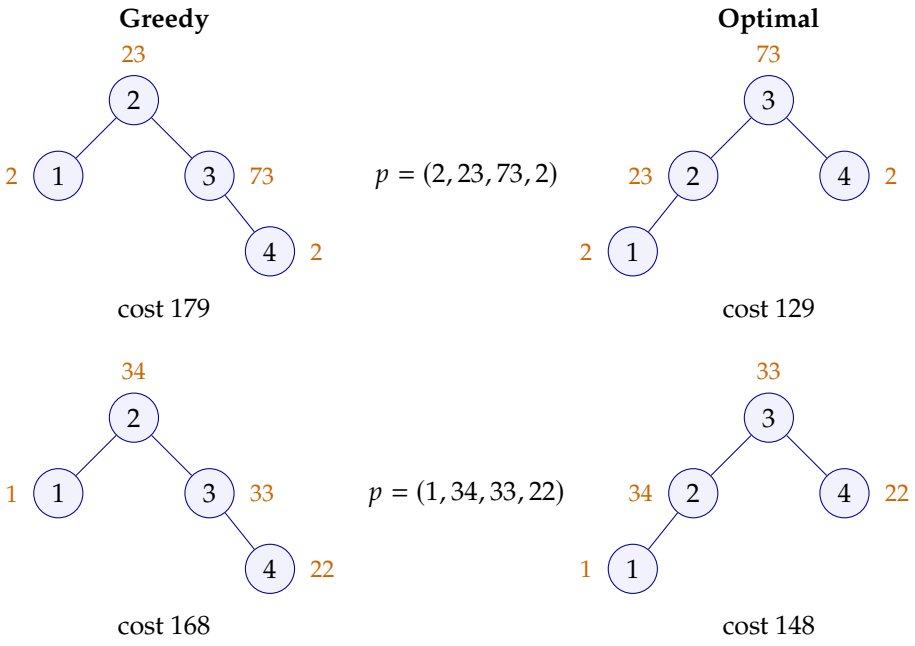


Figure 24: Two four-key instances, with frequencies beside the nodes. In each row, the greedy tree on the left has higher cost than the optimal tree on the right.

With the top-down approach, the choice of root has hard-to-predict repercussions further down the tree; this defeats both greedy and naive divide-and-conquer approaches. But if we knew the root,

the rest of the problem would split into two independent subproblems. A dynamic programming algorithm can try all possible roots.

13.4 Optimal substructure

Lemma 13.2 [Optimal substructure]. *If T is an optimal BST for the keys $\{1, 2, \ldots, n\}$ with root r, then its left subtree T_1 and right subtree T_2 are optimal BSTs for the keys $\{1, \ldots, r-1\}$ and $\{r+1, \ldots, n\}$, respectively.*

The proof rests on a decomposition of the cost. A search for $i < r$ examines the root and then follows the search for i in T_1; similarly for $i > r$ in T_2. Hence

$$\begin{aligned} C(T) &= p_r \cdot 1 + \sum_{i=1}^{r-1} p_i\big(1 + [\text{search time for } i \text{ in } T_1]\big) + \sum_{i=r+1}^{n} p_i\big(1 + [\text{search time for } i \text{ in } T_2]\big) \\ &= \sum_{i=1}^{n} p_i + C(T_1) + C(T_2). \end{aligned}$$

The first term is a constant, independent of the tree.

Proof. Suppose for contradiction that T_1 is not optimal for $\{1, \ldots, r-1\}$, so some BST T_1^* on these keys has $C(T_1^*) < C(T_1)$; the case of T_2 is similar. Obtain T^* from T by cutting out T_1 and pasting in T_1^*. Since every key of T_1^* is less than r, T^* is a valid search tree. By the decomposition,

$$C(T^*) = \sum_{i=1}^{n} p_i + C(T_1^*) + C(T_2) < \sum_{i=1}^{n} p_i + C(T_1) + C(T_2) = C(T),$$

contradicting the optimality of T. □

13.5 Subproblems and recurrence

The items in a subproblem of Lemma 13.2 are a prefix or a suffix of the original items. Applying the lemma again inside such a subproblem produces a prefix or suffix of that range. The relevant subproblems are therefore the contiguous intervals $\{i, i+1, \ldots, j\}$ for $1 \le i \le j \le n$; prefixes alone, or prefixes and suffixes alone, are not enough.

For $1 \le i \le j \le n$, let C_{ij} be the weighted search cost of an optimal BST for the items $\{i, i+1, \ldots, j\}$ with frequencies $p_i, \ldots, p_j$. Interpret $C_{xy} = 0$ if $x > y$. Then, for every $1 \le i \le j \le n$,

$$C_{ij} = \min_{r=i,\ldots,j} \left\{ \sum_{k=i}^{j} p_k + C_{i,r-1} + C_{r+1,j} \right\}.$$

This follows from the decomposition $C(T) = \sum_k p_k + C(T_1) + C(T_2)$: optimal substructure narrows the candidates down to $j - i + 1$ possibilities, one for each root, and the recurrence picks the best by brute force.

13.6 The algorithm

Solve the smallest subproblems, those with the fewest items $j - i + 1$, first. The index $s = j - i$ counts rounds, and $A[i, j]$ holds the optimal value for the items $\{i, \dots, j\}$.

Algorithm 13.1: Optimal binary search tree

Input: Frequencies $p_1, \dots, p_n$ for items $1 < 2 < \cdots < n$.
Output: The minimum weighted search time C_{1n}.

$A \leftarrow$ a 2-D array, with $A[x, y]$ interpreted as 0 whenever $x > y$
for $s \leftarrow 0$ **to** $n - 1$ **do**
  for $i \leftarrow 1$ **to** $n - s$ **do**
    $A[i, i + s] \leftarrow \min_{r=i,\dots,i+s} \left\{ \sum_{k=i}^{i+s} p_k + A[i, r - 1] + A[r + 1, i + s] \right\}$
return $A[1, n]$

Here $i + s$ plays the role of j. The entries $A[i, r - 1]$ and $A[r + 1, i + s]$ concern fewer items, so they were computed in an earlier round and are available for $O(1)$-time lookup. In particular, $A[i, i] = p_i$.

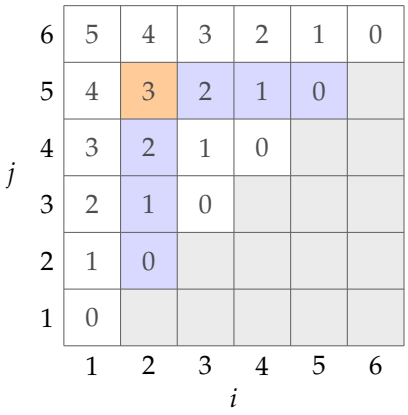


Figure 25: Schematic fill order for $n = 6$, with column i and row j. Each used cell is labelled by its round $s = j - i$; gray cells have $j < i$. The entry $A[2, 5]$ (orange) reads the shaded entries $A[2, r - 1]$ below it and $A[r + 1, 5]$ to its right, all from earlier rounds, together with empty intervals that count as 0.

Proposition 13.3 [Running time]. *The algorithm runs in $\Theta(n^3)$ time.*

Proof. There are $\Theta(n^2)$ subproblems. Computing $A[i, j]$ takes $\Theta(j - i)$ time: one pass for the sum $\sum_{k=i}^{j} p_k$, which is the same for every root, and one pass over the $j - i + 1$ candidate roots. Summing over all intervals gives $\Theta(n^3)$. □

An optimized version of this algorithm, due to Knuth (1971) and Yao (1980), correctly fills the entire

table in only $\Theta(n^2)$ time, that is, $\Theta(1)$ on average per subproblem. It piggybacks on work done in previous subproblems to avoid trying all possible roots.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Optimal Binary Search Trees.

Lecture 14: The Bellman-Ford Algorithm

In the single-source shortest path problem, the input is a directed graph $G = (V, E)$ with n vertices and m edges, a length c_e for each edge $e \in E$, and a source vertex $s \in V$. We may assume there are no parallel edges. The goal is to compute, for every destination $v \in V$, the length (sum of edge lengths) of a shortest s-v path.

Dijkstra's algorithm runs in $O(m \log n)$ time with heaps, but it can fail with negative edge lengths, as when edges represent financial transactions, and it is not naturally distributed, which matters for Internet routing. The Bellman-Ford algorithm addresses both issues.

14.1 Negative cycles

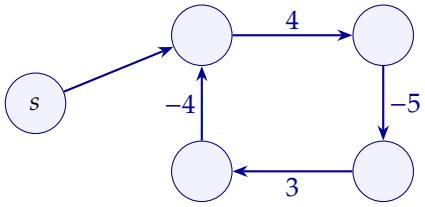


Figure 26: A negative cycle reachable from s. One trip around the cycle has length $4 - 5 + 3 - 4 = -2$, so repeated trips make walks arbitrarily short.

If a negative cycle is reachable from s, as in Figure 26, the notion of a shortest path needs care.

- Allowing cycles makes some distances undefined or $-\infty$, since a walk can keep traversing the negative cycle.
- Requiring cycle-free paths gives an NP-hard problem, so there is no polynomial-time algorithm unless P = NP.
- For now, assume the input graph has no negative cycles. We show later how to check this condition quickly.

Proposition 14.1 [Few edges suffice]. *If G has no negative cycles, then for every v reachable from s there is a shortest s-v path with at most $n - 1$ edges.*

Proof. Take any s-v walk. If it repeats a vertex, it contains a cycle, of nonnegative length, and removing the cycle gives an s-v walk that is no longer. Repeating this yields a cycle-free path that is at least as short. Such a path visits at most n vertices, so it has at most $n - 1$ edges. □

In general, the goal is either (A) to compute the length of a shortest s-v path for every $v \in V$, or (B) to output a negative cycle as the reason for failing to do so.

14.2 Optimal substructure

A subpath of a shortest path should itself be shortest, but in a graph it is not clear which subproblems are smaller than others. The key idea is to restrict the number of edges in a path artificially. The size of a subproblem is the number of permitted edges.

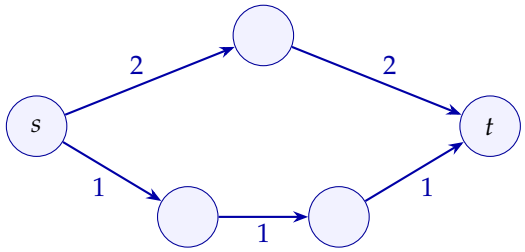


Figure 27: Restricting the number of edges. With at most two edges, the shortest s-t path is the upper one, of length 4. With three edges, the lower path has length 3.

Lemma 14.2 [Optimal substructure]. *Let $G = (V, E)$ be a directed graph with edge lengths c_e and source s, with or without negative cycles. For $v \in V$ and $i \in \{1, 2, \ldots\}$, let P be a shortest s-v path with at most i edges, where cycles are permitted.*

1. *If P has at most $i-1$ edges, then P is a shortest s-v path with at most $i-1$ edges.*
2. *If P has exactly i edges and last hop (w, v), then the prefix P' of P ending at w is a shortest s-w path with at most $i-1$ edges.*

Proof. Case 1: every path with at most $i-1$ edges also has at most i edges, so a shorter one would contradict the optimality of P.
Case 2: suppose an s-w path Q with at most $i-1$ edges is shorter than P'. Then $Q + (w, v)$ is an s-v path with at most i edges, and it is shorter than $P' + (w, v) = P$. This contradicts the optimality of P. □

Thus an optimal solution for destination v has only 1 + in-deg(v) candidates: one from Case 1, and one from Case 2 for each choice of the final hop (w, v).

14.3 The recurrence and the algorithm

Let $L_{i,v}$ be the minimum length of an s-v path with at most i edges, with cycles allowed, and $L_{i,v} = +\infty$ if no such path exists. For every $v \in V$ and $i \geq 1$,

$$L_{i,v} = \min\Big\{ L_{i-1,v},\ \min_{(w,v)\in E} \{L_{i-1,w} + c_{wv}\} \Big\}.$$

This is correct because it is a brute-force search over the $1 + \text{in-deg}(v)$ candidates from Lemma 14.2.

If G has no negative cycles, shortest paths need no cycles, since removing a cycle does not increase the length, and so they have at most $n-1$ edges by Proposition 14.1. It therefore suffices to compute $L_{i,v}$ for $i \in \{0, 1, \ldots, n-1\}$ and all $v \in V$; the answers are the $L_{n-1,v}$.

Algorithm 14.1: Bellman-Ford

Input: Directed graph $G = (V, E)$, edge lengths c_e, source $s \in V$.
Output: Shortest-path distances $A[n-1, v]$ from s, if G has no negative cycle.

$A \leftarrow$ a two-dimensional array indexed by i and v
$A[0, s] \leftarrow 0$; $A[0, v] \leftarrow +\infty$ for all $v \neq s$
for $i \leftarrow 1$ **to** $n-1$ **do**
  for each $v \in V$ **do**
    $A[i, v] \leftarrow \min\{A[i-1, v],\ \min_{(w,v)\in E}\{A[i-1, w] + c_{wv}\}\}$
return $A[n-1, v]$ for all $v \in V$

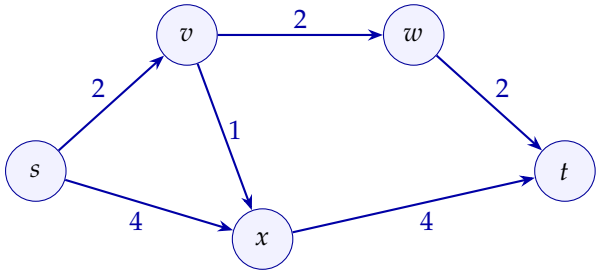


Figure 28: A small input for Bellman-Ford with source s. The rows of $A[i, v]$ appear in the table below.

i	s	v	w	x	t
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	2	$+\infty$	4	$+\infty$
2	0	2	4	3	8
3	0	2	4	3	6
4	0	2	4	3	6

In Figure 28, $A[2, x] = 3$ uses the two-edge path $s \to v \to x$, which beats the direct edge of length 4. At t, the two-edge path through x gives 8, and the three-edge path $s \to v \to w \to t$ gives 6.

Running time. There are $\Theta(n^2)$ subproblems, but one subproblem can take $\Theta(n)$ work, so counting

subproblems gives only $O(n^3)$. Instead, for each i the work is $O\big(\sum_{v \in V} \text{in-deg}(v)\big) = O(m)$, so the total is $O(mn)$.

Stopping early. Row i of A depends only on row $i-1$. If $A[j, v] = A[j-1, v]$ for all v for some $j < n-1$, then all later rows equal row j. In particular $A[n-1, v] = A[j, v]$, and the algorithm can halt. In the table, rows 3 and 4 agree.

14.4 Detecting negative cycles

To report a negative cycle when one exists, run one extra iteration, for $i = n$, still in $O(mn)$ time.

Proposition 14.3 [Negative-cycle check]. *G has no negative cycle reachable from s if and only if $A[n-1, v] = A[n, v]$ for all $v \in V$ in the extended Bellman-Ford algorithm.*

Proof. ($\Rightarrow$) With no reachable negative cycle, removing cycles from an s-v walk never increases its length, so no walk is shorter than a shortest path, which has at most $n-1$ edges. Hence $A[n-1, v]$ and $A[n, v]$ both equal the distance.
($\Leftarrow$) Suppose $A[n-1, v] = A[n, v]$ for all v, and call the common value $d(v)$. Consider a cycle C reachable from s; its vertices have finite values d. By the recurrence,

$$d(v) = A[n, v] \le A[n-1, w] + c_{wv} = d(w) + c_{wv} \qquad \text{for every edge } (w, v) \in E,$$

that is, $c_{wv} \ge d(v) - d(w)$. Summing over the edges of C, the right side telescopes:

$$\sum_{(w,v) \in C} c_{wv} \;\ge\; \sum_{(w,v) \in C} \big(d(v) - d(w)\big) = 0.$$

So no reachable cycle is negative. □

14.5 Space and predecessor pointers

The basic algorithm stores $\Theta(n^2)$ values, $\Theta(1)$ for each of n^2 subproblems. Since row i uses only row $i-1$, it is enough to keep the current and previous rounds, which takes $O(n)$ space, $O(1)$ per destination.

Without the full table, paths are recovered with a second table of predecessor pointers: $B[i, v]$ is the second-to-last vertex on a shortest s-v path with at most i edges, or NULL if no such path exists. Set $B[0, v]$ = NULL for all v. For $i > 0$:

- if Case 1 of the recurrence wins, $B[i, v] = B[i-1, v]$;
- if Case 2 wins, $B[i, v]$ is the vertex w achieving the minimum, that is, the new last hop.

Thus $B[i, v]$ caches the last hop of the winner in the brute-force search for $A[i, v]$.

If G has no negative cycles, $B[n-1, v]$ is the last hop of a shortest s-v path. Tracing the pointers back from v to s yields a shortest s-v path, by optimal substructure. To find a negative cycle instead, check after each round, with depth-first search, whether the predecessor pointers form a cycle; such a cycle must have negative cost.

14.6 From Bellman-Ford to Internet routing

Bellman-Ford is naturally distributed: each vertex updates its value using only the values of its neighbors. Three changes lead to a routing protocol.

Destination-driven. Reverse all edge directions. Each vertex v stores its shortest-path distance to a destination t and the first hop of a shortest path, for every relevant destination t. These are DISTANCE VECTOR PROTOCOLS.

Asynchrony. One cannot assume that all values of round $i-1$ are computed before any of round i. The fix is to switch from pull-based to push-based updates: as soon as the estimate at v decreases, v notifies all of its neighbors. Assuming no negative cycles, this is guaranteed to converge eventually, because updates strictly decrease the sum of the shortest-path estimates. The RIP and RIP2 Internet routing protocols are very close to this algorithm (RFC 1058).

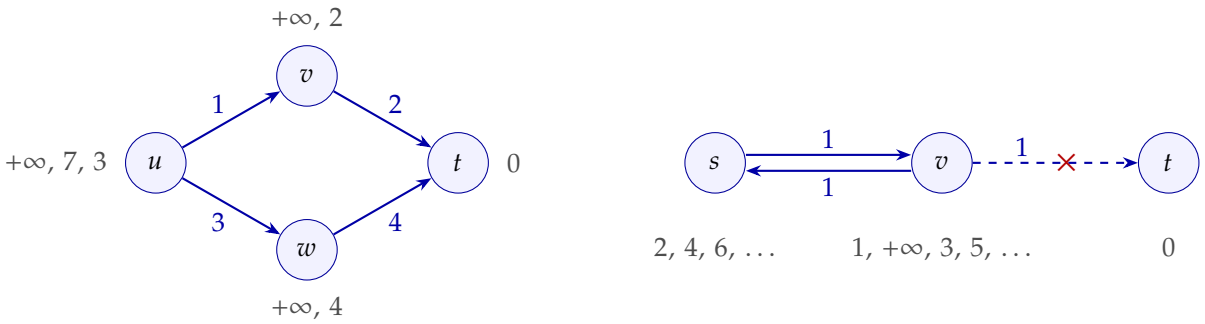


Figure 29: Distance estimates to t. Left: with asynchronous updates, u may hear from w (estimate 7) before v (estimate 3). Right: after (v, t) fails, s and v keep raising each other's estimates.

Failures. Convergence is guaranteed only for static networks. After a link fails, estimates can COUNT TO INFINITY, as in the right panel of Figure 29: when (v, t) fails, v learns from s a route of length 2 (which actually passes through v), sets its estimate to 3, then s updates to 4, and so on without bound. The fix is for each vertex to maintain the entire shortest path to t, not just the next hop. This is a PATH VECTOR PROTOCOL, used by the Border Gateway Protocol (BGP). It needs more space, but it is more robust to failures and permits more sophisticated route selection, such as caring about intermediate stops.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), The Bellman-Ford Algorithm.

Lecture 15: All-Pairs Shortest Paths

In the all-pairs shortest path problem (APSP), the input is a directed graph $G = (V, E)$ with n vertices, m edges, and an edge cost c_e for each $e \in E$; there is no distinguished source. The goal is either (A) to compute the length of a shortest u-v path for all pairs $u, v \in V$, or (B) to report correctly that G contains a negative cycle.

15.1 Reduction to single-source shortest paths

Running a single-source subroutine once from each vertex, n invocations in all, solves APSP.

- With nonnegative edge costs, n runs of Dijkstra's algorithm take $O(nm \log n)$ time: $O(n^2 \log n)$ if $m = \Theta(n)$ and $O(n^3 \log n)$ if $m = \Theta(n^2)$.
- With general edge costs, n runs of Bellman-Ford take $O(n^2 m)$ time: $O(n^3)$ if $m = \Theta(n)$ and $O(n^4)$ if $m = \Theta(n^2)$.

The Floyd-Warshall algorithm solves APSP in $O(n^3)$ time, even with negative edge lengths. It is therefore at least as good as n runs of Bellman-Ford and better in dense graphs. With nonnegative edge costs, it is competitive with n runs of Dijkstra's algorithm in dense graphs. An important special case is the transitive closure of a binary relation, that is, all-pairs reachability. Whether APSP can be solved significantly faster than $O(n^3)$ in dense graphs is an open question.

15.2 Optimal substructure

Ordering subproblems in a graph problem can be tricky. The key idea is to order the vertices arbitrarily as $V = \{1, 2, \ldots, n\}$ and to let $V^{(k)} = \{1, 2, \ldots, k\}$. Subproblems restrict which vertices may appear in the interior of a path.

Lemma 15.1 [Optimal substructure]. *Suppose G has no negative cycle. Fix a source $i \in V$, a destination $j \in V$, and $k \in \{1, 2, \ldots, n\}$. Let P be a shortest cycle-free i-j path with all internal vertices in $V^{(k)}$.*

1. *If k is not internal to P, then P is a shortest cycle-free i-j path with all internal vertices in $V^{(k-1)}$.*
2. *If k is internal to P, then P splits at k into P_1 and P_2, where P_1 is a shortest cycle-free i-k path and P_2 is a shortest cycle-free k-j path, each with all internal vertices in $V^{(k-1)}$.*

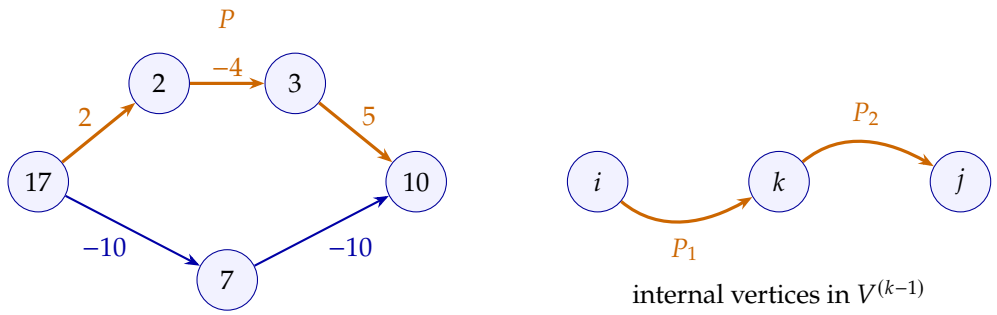


Figure 30: Left: with $i = 17$, $j = 10$, and $k = 5$, the path P through 2 and 3 is a shortest i-j path with internal vertices in $V^{(5)}$; the shorter path through 7 is not allowed. Right: in Case 2, P passes through k once and splits into P_1 and P_2.

Proof. Case 1: P is a candidate for the smaller subproblem, and every candidate for the smaller subproblem is also one for the larger. So P is optimal for both.
Case 2: since P is cycle-free, k appears on it exactly once. So P_1 and P_2 are cycle-free with all internal vertices in $V^{(k-1)}$, and the length of P is the sum of their lengths. Suppose a cycle-free i-k path Q_1 with internal vertices in $V^{(k-1)}$ is shorter than P_1. Then $Q_1 + P_2$ is an i-j walk with internal vertices in $V^{(k)}$ that is shorter than P. It may repeat a vertex, but deleting its cycles, none of which is negative, gives a cycle-free i-j path with internal vertices in $V^{(k)}$ that is still shorter than P, a contradiction. The argument for P_2 is symmetric. □

15.3 The Floyd-Warshall algorithm

Let $A[i, j, k]$ be the length of a shortest i-j path with all internal vertices in $\{1, 2, \ldots, k\}$, or $+\infty$ if no such path exists. For $k = 0$ no internal vertices are allowed, so

$$A[i,j,0] = \begin{cases} 0 & \text{if } i = j, \\ c_{ij} & \text{if } (i,j) \in E, \\ +\infty & \text{if } i \neq j \text{ and } (i,j) \notin E. \end{cases}$$

Algorithm 15.1: Floyd-Warshall

Input: Directed graph $G = (V, E)$ with $V = \{1, \ldots, n\}$ and edge costs c_e.
Output: Shortest-path distances $A[i, j, n]$ for all $i, j \in V$.

```
A ← a three-dimensional array indexed by i, j, k
Set A[i, j, 0] for all i, j ∈ V by the base cases
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            A[i, j, k] ← min{A[i, j, k − 1], A[i, k, k − 1] + A[k, j, k − 1]}
return A[i, j, n] for all i, j ∈ V
```

The two terms are Case 1 and Case 2 of Lemma 15.1, so if G has no negative cycle, correctness follows by induction on k. Each subproblem takes $O(1)$ time, so the total is $O(n^3)$.

Negative cycles. If G has a negative cycle, then $A[i, i, n] < 0$ for at least one $i \in V$ at the end of the algorithm. If G has no negative cycle, every $A[i, i, n]$ equals the length 0 of the empty i-i path, so checking the diagonal detects negative cycles.

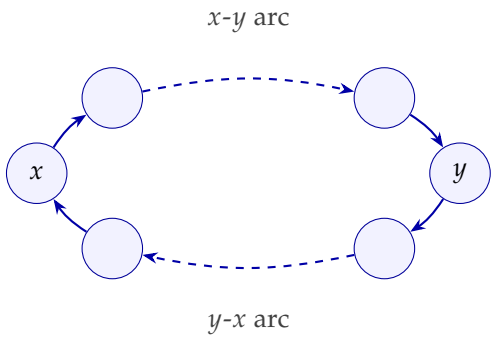


Figure 31: A negative cycle through x, where y has the largest label on the cycle. Both arcs have internal vertices with labels below y.

Proof of the negative-cycle check. By induction on k, $A[i, j, k]$ is at most the length of every cycle-free i-j path with internal vertices in $V^{(k)}$, even if G has negative cycles: if the path avoids k, use $A[i, j, k-1]$; otherwise split it at its single visit to k. Also, $A[i, j, k] \le A[i, j, k-1]$.
Let C be a negative cycle that repeats no vertex and has at least two vertices, let y be the vertex of C with the largest label, and let $x \neq y$ be another vertex of C, as in Figure 31. The cycle consists of an x-y arc and a y-x arc, both cycle-free with internal labels below y. Hence

$$A[x, x, n] \le A[x, x, y] \le A[x, y, y-1] + A[y, x, y-1] \le (\text{length of } C) < 0. \qquad \square$$

Reconstructing paths. Along with A, compute $B[i, j]$, the largest label of an internal vertex on a shortest i-j path. Whenever Case 2 of the recurrence is used to compute $A[i, j, k]$, reset $B[i, j] = k$. A shortest i-j path then consists of a shortest i-$B[i, j]$ path followed by a shortest $B[i, j]$-j path, each reconstructed recursively.

15.4 Reweighting

Johnson's algorithm reduces APSP to one invocation of Bellman-Ford, taking $O(mn)$ time, and n invocations of Dijkstra's algorithm, taking $O(nm \log n)$ time. The total, $O(mn) + O(mn \log n) = O(mn \log n)$, is as good as for nonnegative edge lengths.

Adding the same constant M to every edge length does not preserve shortest paths unless all s-t paths have the same number of edges. For example, if $s \to v \to t$ has edges of length 1 and 1 and the direct edge (s, t) has length 3, the two-edge path is shorter. After adding $M = 2$, the lengths

become 6 and 5, and the direct edge is shorter.

Instead, fix a real number p_v for each vertex v and define the REWEIGHTED LENGTH

$$c'_e = c_e + p_u - p_v \qquad \text{for each edge } e = (u, v).$$

For example, an edge of length 2 with $p_u = -4$ and $p_v = -3$ gets length $2 + (-4) - (-3) = 1$.

Proposition 15.2 [REWEIGHTING PRESERVES SHORTEST PATHS]. *If an s-t path P has length L under the lengths c_e, then its length under the lengths c'_e is $L + p_s - p_t$.*

Proof. The sum telescopes along the path:

$$\sum_{e \in P} c'_e = \sum_{e=(u,v) \in P} (c_e + p_u - p_v) = \left(\sum_{e \in P} c_e\right) + p_s - p_t. \qquad \square$$

Every s-t path shifts by the same amount $p_s - p_t$, so reweighting never changes which s-t paths are shortest. This is useful if G has some negative edge lengths but, after reweighting by suitable $\{p_v\}$, all edge lengths become nonnegative. Such weights always exist when G has no negative cycle, and Bellman-Ford computes them. Bellman-Ford is then needed once, and it enables Dijkstra's algorithm.

15.5 Johnson's algorithm

Add a new vertex s with an edge of length 0 to every vertex of G, and let p_v be the length of a shortest s-v path in this larger graph. Since s has no incoming edges, adding it creates no new u-v paths for u, v in G.

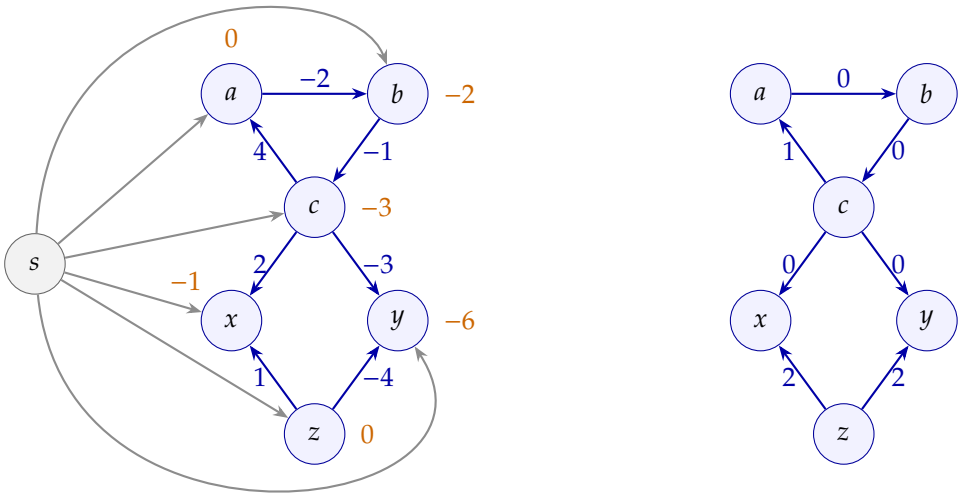


Figure 32: Left: G', formed by adding s with an edge of length 0 (gray) to every vertex; the numbers beside the vertices are the potentials p_v. Right: the reweighted lengths $c'_e = c_e + p_u - p_v$ on G, all nonnegative.

In Figure 32, the potentials are $p_a = 0$, $p_b = -2$, $p_c = -3$, $p_x = -1$, $p_y = -6$, and $p_z = 0$. After reweighting, every edge length is 0, 1, or 2, so all reweighted shortest paths can be computed with n runs of Dijkstra's algorithm, with no further need for Bellman-Ford.

Algorithm 15.2: Johnson's algorithm

Input: Directed graph $G = (V, E)$ with general edge lengths c_e.
Output: Shortest-path distances $d(u, v)$ for all $u, v \in V$, or a negative cycle.

Form G' by adding a new vertex s and an edge (s, v) of length 0 for each $v \in V$
Run Bellman-Ford on G' from s
if Bellman-Ford detects a negative cycle in G' **then**
    Halt and report the negative cycle, which must lie in G
$p_v \leftarrow$ length of a shortest s-v path in G', for each $v \in V$
$c'_e \leftarrow c_e + p_u - p_v$, for each $e = (u, v) \in E$
for each $u \in V$ **do**
    Run Dijkstra's algorithm on G from u with lengths c'_e, giving $d'(u, v)$ for all $v \in V$
return $d(u, v) \leftarrow d'(u, v) - p_u + p_v$ for each pair $u, v \in V$

Running time. Forming G' takes $O(n)$, Bellman-Ford $O(mn)$, forming c' $O(m)$, the n runs of Dijkstra's algorithm $O(nm \log n)$, and the final step $O(1)$ per pair, $O(n^2)$ in all. The total is $O(mn \log n)$, much better than Floyd-Warshall for sparse graphs.

Correctness. Given that every $c'_e \geq 0$, Dijkstra's algorithm correctly computes the reweighted distances. By Proposition 15.2, reweighting does not change the shortest u-v path; it only adds $p_u - p_v$ to its length. So $d(u, v) = d'(u, v) - p_u + p_v$.

Proposition 15.3 [Nonnegative reweighted lengths]. *For every edge $e = (u, v)$ of G, the reweighted length $c'_e = c_e + p_u - p_v$ is nonnegative.*

Proof. By construction, p_u and p_v are the lengths of shortest s-u and s-v paths in G'; they exist because of the edges from s. Let P be a shortest s-u path in G', of length p_u. Then $P + (u, v)$ is an s-v path of length $p_u + c_{uv}$. A shortest s-v path is no longer, so $p_v \leq p_u + c_{uv}$, that is, $c'_{uv} = c_{uv} + p_u - p_v \geq 0$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), All-Pairs Shortest Paths.

Lecture 16: NP-Complete Problems

Many important computational problems seem impossible to solve efficiently. NP-completeness gives a precise way to express this intractability, and its conclusion shapes which algorithmic goals are reasonable.

16.1 Polynomial-time solvability and the class P

Measure the input size n by the input length: the number of keystrokes needed to describe the input.

Definition 16.1 [Polynomial-time solvable]. A problem is polynomial-time solvable if some algorithm correctly solves it in $O(n^k)$ time, for some constant k. The class P is the set of polynomial-time solvable problems.

Any constant exponent qualifies, even $k = 10{,}000$. The definition uses deterministic algorithms; allowing randomization does not change the picture to first order. P is a rough litmus test for computational tractability.

Almost every problem solved so far in the course is in P. Two exceptions are cycle-free shortest paths in graphs with negative cycles, and knapsack. The knapsack dynamic program runs in $\Theta(nW)$ time, but the capacity W is written with about $\log W$ digits, so the input length is proportional to $\log W$ rather than W. Thus nW can be exponential in the input length. Both problems are NP-complete.

Definition 16.2 [Traveling salesman problem]. The input is a complete undirected graph with nonnegative edge costs. The output is a minimum-cost tour: a cycle that visits every vertex exactly once. This is the traveling salesman problem (TSP).

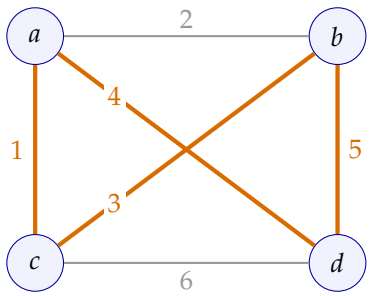


Figure 33: A four-vertex TSP instance. The highlighted tour $a \to c \to b \to d \to a$ costs $1 + 3 + 5 + 4 = 13$, the optimum. The other two tours cost $2 + 5 + 6 + 1 = 14$ and $2 + 3 + 6 + 4 = 15$.

Note. Edmonds' conjecture (1965): there is no polynomial-time algorithm for TSP. Since TSP is NP-complete (see below), this is equivalent to P ≠ NP.

16.2 Reductions and completeness

Instead of proving outright that TSP is hard, amass evidence through relative difficulty: show that TSP is at least as hard as many other problems.

Definition 16.3 [Reduction]. A problem Π_1 reduces to a problem Π_2 if, given a polynomial-time subroutine for Π_2, we can use it to solve Π_1 in polynomial time.

For example, computing the median reduces to sorting (sort, then read the middle entry); detecting a cycle reduces to depth-first search; and all-pairs shortest paths reduces to single-source shortest paths (one call per source vertex).

Proposition 16.4 [Reductions transfer tractability]. *Suppose* Π_1 *reduces to* Π_2. *If* $\Pi_2 \in \mathrm{P}$, *then* $\Pi_1 \in \mathrm{P}$. *Equivalently, if* $\Pi_1 \notin \mathrm{P}$, *then* $\Pi_2 \notin \mathrm{P}$.

Proof. Replace the subroutine in the reduction by a polynomial-time algorithm for Π_2. The reduction makes polynomially many calls, each on an input of polynomial length, and a polynomial of a polynomial is a polynomial. The second statement is the contrapositive. □

So if Π_1 reduces to Π_2, then Π_2 is at least as hard as Π_1.

Definition 16.5 [Completeness]. Let C be a set of problems. A problem Π is C-complete if $\Pi \in C$ and every problem in C reduces to Π.

A C-complete problem is a hardest problem in C. The plan is to show that TSP is C-complete for a very large class C. The class of all problems is too large. In the halting problem, the input is a program and an input for it, and the question is whether the program eventually halts. By Turing's theorem (1936), no algorithm, however slow, solves it. TSP, in contrast, is solvable in finite time by brute-force search, so the halting problem cannot reduce to TSP. The refined goal is to show that TSP is as hard as every problem solvable by brute-force search.

16.3 The class NP and NP-completeness

Definition 16.6 [NP]. A problem is in NP if

1. its solutions always have length polynomial in the input size, and
2. purported solutions can be verified in polynomial time.

Examples are deciding whether a TSP instance has a tour of length at most 1000, and constraint satisfaction problems such as 3SAT. A NP-complete problem is a C-complete problem for C = NP.

Every problem in NP can be solved by brute-force search in exponential time: there are exponentially many candidate solutions of polynomial length, and each can be checked in polynomial time. The vast majority of natural computational problems are in NP; roughly, being in NP means that a solution can be recognized efficiently.

Proposition 16.7 [One NP-complete problem in P suffices]. *If some NP-complete problem* Π *is polynomial-time solvable, then every problem in NP is polynomial-time solvable; that is,* P = NP.

Proof. Every problem in NP reduces to Π, so Proposition 16.4 applies to each of them. □

Hence NP-completeness is strong evidence of intractability. An NP-complete problem simultaneously encodes *all* problems whose solutions can be efficiently recognized; it is a universal problem. Two facts make the theory useful. NP-complete problems exist (Cook 1971, Levin 1973). Moreover, starting with Karp (1972), thousands of natural and important problems, including TSP, have been shown to be NP-complete.

Proposition 16.8 [Proving NP-completeness]. *Let* Π' *be a known NP-complete problem (for instance, from Garey and Johnson,* Computers and Intractability*). If* $\Pi \in$ NP *and* Π' *reduces to* Π*, then* Π *is NP-complete.*

Proof. Reductions compose: a reduction from a problem Π'' to Π' that calls a subroutine for Π' can implement each call through the reduction from Π' to Π, and the result still runs in polynomial time. Every problem in NP reduces to Π', and hence to Π. Together with $\Pi \in$ NP, this is the definition of NP-completeness. □

The reduction shows that Π is at least as hard as Π'; membership in NP is checked separately.

16.4 The P versus NP question

The central question is whether P = NP: is every problem whose solutions can be verified in polynomial time also solvable in polynomial time? It is widely conjectured that P ≠ NP, but this has not been proved; it is one of the Clay Institute's million-dollar problems. Reasons to believe the conjecture are psychological (if P = NP, someone would likely have proved it by now) and philosophical (if P = NP, finding a proof would always be as easy as verifying one). There is no convincing mathematical reason yet.

The name NP stands for *nondeterministic polynomial*, not "not polynomial." The definition by efficient verification of purported solutions is a modern, mathematically equivalent formulation.

16.5 Algorithmic approaches to NP-complete problems

NP-completeness is not a death sentence for a problem, but it calls for appropriate expectations and strategy. Three strategies are useful.

1. **Focus on computationally tractable special cases.** Maximum-weight independent set is NP-complete in general graphs but solvable in path graphs, and also in trees and graphs of bounded tree width. Knapsack is tractable when the capacity is polynomial in size, for example $W = O(n)$. 2SAT is in P, while 3SAT is NP-complete. Vertex cover is tractable when the optimal cover is small.
2. **Heuristics:** fast algorithms that are not always correct. Lecture 18 gives greedy and dynamic-programming heuristics for knapsack.
3. **Exponential time, but faster than brute-force search.** Knapsack takes $O(nW)$ time rather than the 2^n of checking every subset. Lecture 17 solves TSP in roughly 2^n rather than $n!$ time, and vertex cover in roughly $2^{\text{OPT}}n$ rather than n^{OPT} time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), NP-Complete Problems.

Lecture 17: Faster Exact Algorithms for NP-Complete Problems

An NP-complete problem can still be solved exactly in exponential time that is far smaller than brute-force search. This lecture gives two examples: a search algorithm for small vertex covers and a dynamic program for TSP.

17.1 The vertex cover problem

Definition 17.1 [Vertex cover]. Let $G = (V, E)$ be an undirected graph. A vertex cover is a subset $S \subseteq V$ that contains at least one endpoint of each edge of G. The vertex cover problem asks for a vertex cover of minimum cardinality.

Example: In a star graph with n vertices, the center alone is a vertex cover, so the minimum size is 1. In a clique with n vertices, any two vertices outside a cover would leave the edge between them uncovered. Hence at most one vertex is omitted, and the minimum size is $n - 1$.

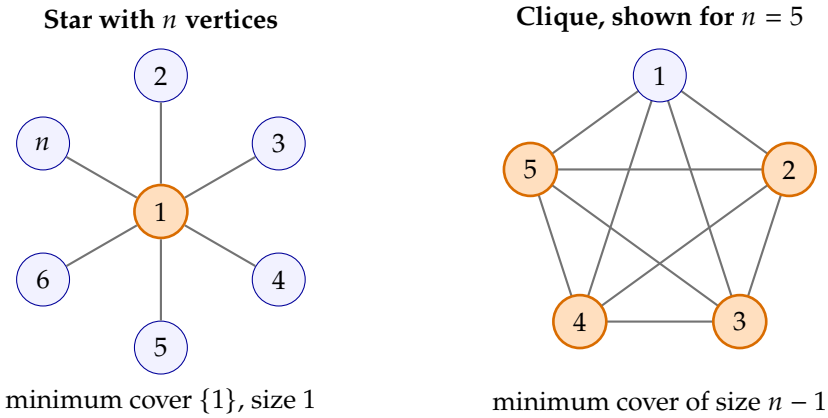


Figure 34: Minimum vertex covers (highlighted) of a star and a clique.

In general, vertex cover is NP-complete. Following the three strategies of Lecture 16:

1. Tractable special cases include trees (by dynamic programming), bipartite graphs (by the maximum flow problem), and graphs whose optimal cover is small, roughly $\log n$ or less.
2. Heuristics, for example suitable greedy algorithms.
3. Exponential time that improves on brute-force search, developed next.

17.2 Smarter search for small vertex covers

Given a positive integer k, thought of as small, decide whether G has a vertex cover of size at most k. Trying all subsets of size k takes about $\binom{n}{k} = \Theta(n^k)$ time. A better search branches on the endpoints of a single edge.

For a vertex u, let G_u be G with u and its incident edges deleted. Write $E = E_u \cup F_u$, where E_u is the edge set of G_u and F_u consists of the edges incident to u.

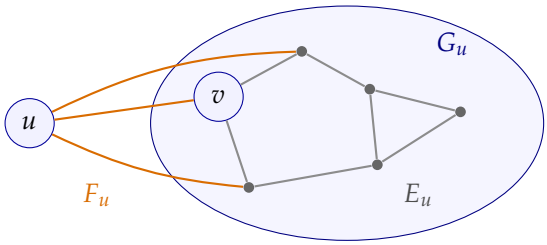


Figure 35: Deleting u leaves the graph G_u with edge set E_u. The deleted edges F_u, including (u, v), are covered by u.

Lemma 17.2 [Substructure lemma]. *Let $(u, v) \in E$ and $k \geq 1$. Then G has a vertex cover of size at most k if and only if G_u or G_v (or both) has a vertex cover of size at most $k - 1$.*

Proof. ($\Leftarrow$) Suppose G_u, say, has a vertex cover S with $|S| \leq k - 1$. Then S contains an endpoint of each edge of E_u, and u is an endpoint of each edge of F_u. So $S \cup \{u\}$ is a vertex cover of G of size at most k.
($\Rightarrow$) Let S be a vertex cover of G with $|S| \leq k$. Since (u, v) is an edge, S contains u or v, say u. No edge of E_u is incident to u, so each edge of E_u has an endpoint in $S - \{u\}$. Thus $S - \{u\}$ is a vertex cover of G_u of size at most $k - 1$. □

Algorithm 17.1: Search for a vertex cover of size at most k

Input: Undirected graph $G = (V, E)$ and integer $k \geq 0$.
Output: A vertex cover of size at most k, or FAIL if none exists.

```
if E = ∅ then
    return ∅
if k = 0 then
    return FAIL
Pick an arbitrary edge (u, v) ∈ E
S ← Search(G_u, k − 1)
if S ≠ FAIL then
    return S ∪ {u}
S ← Search(G_v, k − 1)
if S ≠ FAIL then
    return S ∪ {v}
return FAIL
```

Theorem 17.3 [Vertex cover search]. *The search algorithm is correct and runs in $O(2^k m)$ time, where $m = |E|$.*

Proof. Correctness follows by induction on k. If $E = \emptyset$, the empty set is a cover. If $E \neq \emptyset$ and $k = 0$, no cover of size 0 exists. Otherwise, by the induction hypothesis the two recursive calls correctly decide whether G_u or G_v has a cover of size at most $k - 1$, and a returned set extends to a cover of G by the ($\Leftarrow$) direction of Lemma 17.2. If both calls fail, the ($\Rightarrow$) direction shows that G has no cover of size at most k.
For the running time, each call makes at most two recursive calls, each with parameter decreased by one, and calls with $k = 0$ make none. The recursion tree therefore has branching factor at most 2 and depth at most k, so it has at most $2^{k+1} - 1 = O(2^k)$ nodes. Each call does $O(m)$ work outside its recursive subcalls, for choosing an edge and forming G_u and G_v. The total is $O(2^k m)$. □

This bound is polynomial whenever $k = O(\log n)$, remains feasible even when $k \approx 20$, and is far better than $\Theta(n^k)$.

17.3 The traveling salesman problem

Recall TSP from Lecture 16: given a complete undirected graph with vertices $\{1, 2, \ldots, n\}$ and nonnegative edge costs c_{ij}, find a minimum-cost tour. Brute-force search over tours takes about $n!$ time, which is tractable only for $n \approx 12$ or 13. Dynamic programming achieves $O(n^2 2^n)$ time, which is tractable for n close to 30.

A natural first attempt copies the format of the Bellman–Ford algorithm. Each candidate below indexes subproblems by an edge budget $i \in \{0, 1, \ldots, n\}$ and a destination $j \in \{1, 2, \ldots, n\}$, with L_{ij} a shortest-path length from vertex 1 to j.

1. **At most i edges.** These subproblems are polynomial in number and easy to compute, but solving all of them does not solve TSP: nothing forces a path to visit every vertex.
2. **Exactly i edges.** Solving these still does not solve TSP: a path with exactly n edges may repeat vertices and skip others.
3. **Exactly i edges and no repeated vertices.** There are only polynomially many subproblems, and they would solve TSP: the optimal tour cost is $\min_j \{L_{n-1,j} + c_{j1}\}$. The obstacle is that bigger subproblems cannot be computed efficiently from smaller ones.

For the third attempt, one would hope to use the recurrence

$$L_{ij} = \min_{k \neq 1, j} \{L_{i-1,k} + c_{kj}\},$$

combining a shortest path from 1 to k with $i - 1$ edges and no repeated vertices with the final

hop (k, j). But j may already appear on that $1 \to k$ path, and appending (k, j) would then visit j twice, which is not allowed. To enforce that each vertex is visited exactly once, a subproblem must remember the *identities* of the vertices it has visited, though not the order in which they were visited.

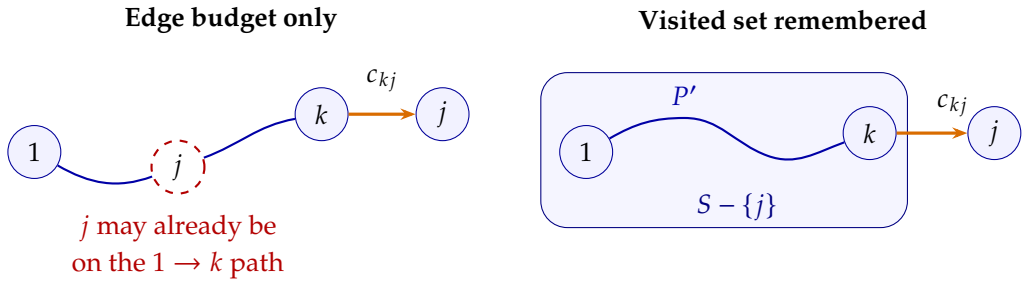


Figure 36: Left: with only an edge budget, the best $1 \to k$ path may already pass through j, so appending (k, j) revisits j. Right: a subproblem that fixes the visited set $S - \{j\}$ excludes j from P'.

17.4 A dynamic programming algorithm for TSP

Definition 17.4 [TSP SUBPROBLEMS]**.** For every destination $j \in \{1, 2, \ldots, n\}$ and every subset $S \subseteq \{1, 2, \ldots, n\}$ that contains 1 and j, let $L_{S,j}$ be the minimum length of a path from 1 to j that visits precisely the vertices of S, each exactly once.

The size of a subproblem is $|S|$.

Lemma 17.5 [OPTIMAL SUBSTRUCTURE]**.** *Let $|S| \geq 2$ and let P be a shortest path from 1 to j that visits each vertex of S exactly once. If the last hop of P is (k, j), then the rest of the path, P', is a shortest path from 1 to k that visits each vertex of $S - \{j\}$ exactly once.*

Proof. Use cut and paste. P' visits exactly the vertices of $S - \{j\}$ and ends at k. If some path Q from 1 to k visiting each vertex of $S - \{j\}$ exactly once were shorter than P', then Q followed by (k, j) would visit each vertex of S exactly once, because $j \notin S - \{j\}$, and would be shorter than P. This contradicts the choice of P. □

Conversely, for every $k \in S - \{j\}$, an optimal path for $(S - \{j\}, k)$ followed by (k, j) is a valid path for (S, j). Hence

$$L_{S,j} = \min_{k \in S,\, k \neq j} \{L_{S-\{j\},k} + c_{kj}\}.$$

A path from 1 to itself cannot visit additional vertices without returning to 1 twice, so $L_{S,1} = 0$ if $S = \{1\}$ and $+\infty$ otherwise.

Algorithm 17.2: Dynamic programming for TSP

Input: Complete undirected graph on $\{1, \dots, n\}$, $n \geq 3$, with costs $c_{ij} \geq 0$.
Output: The minimum cost of a tour.

for each $S \subseteq \{1, \dots, n\}$ with $1 \in S$ **do**
  $A[S, 1] \leftarrow 0$ if $S = \{1\}$, and $A[S, 1] \leftarrow +\infty$ otherwise
for $m \leftarrow 2$ **to** n **do**
  for each $S \subseteq \{1, \dots, n\}$ with $|S| = m$ and $1 \in S$ **do**
    for each $j \in S$ with $j \neq 1$ **do**
      $A[S, j] \leftarrow \min_{k \in S,\, k \neq j} \{A[S - \{j\}, k] + c_{kj}\}$
return $\min_{j=2,\dots,n} \{A[\{1, \dots, n\}, j] + c_{j1}\}$

Subsets are processed in order of size, so each $A[S - \{j\}, k]$ is available when needed. In the final line, $A[\{1, \dots, n\}, j]$ is the minimum cost of a path from 1 to j visiting every vertex once, and c_{j1} is the cost of the final hop that closes the tour. Every tour is such a path followed by its return edge, so the minimum is the optimal tour cost.

Theorem 17.6 [TSP RUNNING TIME**].** *The algorithm runs in $O(n^2 2^n)$ time.*

Proof. There are at most n choices of j and 2^{n-1} choices of S, so $O(n2^n)$ subproblems. Each takes $O(n)$ time, a minimum over at most n choices of k. The final line takes $O(n)$ time. The total is $O(n2^n) \cdot O(n) = O(n^2 2^n)$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Faster Exact Algorithms for NP-Complete Problems.

Lecture 18: Approximation Algorithms for NP-Complete Problems

A heuristic for an NP-complete problem should ideally come with a performance guarantee: a proof that its output is almost correct on all, or at least many, instances. For knapsack, this lecture gives a greedy heuristic that is always within a factor of two of optimal, and a dynamic-programming heuristic whose accuracy can be made arbitrarily good.

18.1 Knapsack and a greedy heuristic

Definition 18.1 [Knapsack]. The input is n items, where item i has a positive value v_i and a size w_i, together with a knapsack capacity W. The output is a subset $S \subseteq \{1, 2, \ldots, n\}$ that

$$\text{maximizes } \sum_{i \in S} v_i \quad \text{subject to} \quad \sum_{i \in S} w_i \leq W.$$

Assume every item fits on its own, $w_i \leq W$; any other item can be discarded in advance. Ideal items have big value and small size, which suggests ordering items by their value per unit size, or bang-per-buck, v_i/w_i.

Algorithm 18.1: Three-step greedy knapsack heuristic

Input: Values $v_i > 0$, sizes $0 < w_i \leq W$, and capacity W.
Output: A feasible subset of items.

1 Sort and reindex the items so that $v_1/w_1 \geq v_2/w_2 \geq \cdots \geq v_n/w_n$
2 $S \leftarrow \emptyset$
3 **for** $i \leftarrow 1$ **to** n **do**
4 **if** $\sum_{\ell \in S} w_\ell + w_i > W$ **then break**
5 $S \leftarrow S \cup \{i\}$
6 Let i^* be an item of maximum value
7 **return** the more valuable of S and $\{i^*\}$

Steps 1 and 2 sort, then pack items in order until one does not fit. With $W = 5$ and $(v_i, w_i) = (2, 1), (4, 3), (3, 3)$, they return $\{1, 2\}$, which is optimal. Alone, however, they can be arbitrarily bad: with $W = 1000$ and items $(v_1, w_1) = (2, 1)$, $(v_2, w_2) = (1000, 1000)$, they return item 1 of value 2, whereas the optimum takes item 2 with value 1000. Step 3 fixes this by also considering the single most valuable item.

Theorem 18.2 [Greedy approximation]. *The three-step greedy heuristic runs in $O(n \log n)$ time, and its solution always has value at least 50% of the optimal value. That is, it is a $\frac{1}{2}$-approximation algorithm.*

Sorting dominates the running time; the accuracy guarantee is proved next.

18.2 Analysis of the greedy heuristic

If Step 2 packs every item, its solution is optimal. Otherwise, suppose Step 2 packs items $1, \dots, k$ and item $k+1$ does not fit. As a thought experiment, fill the knapsack completely by also packing the fraction

$$\alpha = \frac{W - \sum_{i \le k} w_i}{w_{k+1}} \in [0, 1)$$

of item $k+1$, with value pro-rated to αv_{k+1}. Call this the GREEDY FRACTIONAL SOLUTION, and let $F = \sum_{i \le k} v_i + \alpha v_{k+1}$ be its value.

Example: With $W = 3$, $v_1 = 3$, $v_2 = 2$, and $w_1 = w_2 = 2$, the greedy fractional solution takes all of item 1 and half of item 2, so $F = 3 + 1 = 4$. Only one item fits in a genuine solution, so the optimal value is $3 < F$.

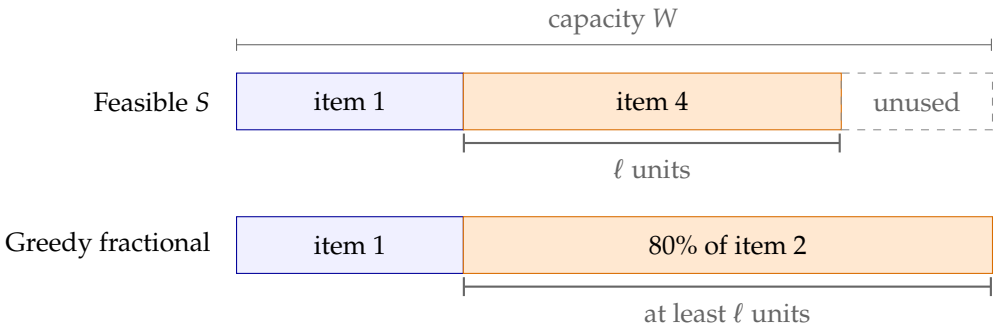


Figure 37: Schematic exchange in the proof of Lemma 18.3. The ℓ units that S spends on item 4 are matched by at least ℓ units that the greedy fractional solution spends on item 2, which has a larger bang-per-buck. Widths are illustrative.

Lemma 18.3 [FRACTIONAL BOUND]. *The greedy fractional solution is at least as good as every feasible non-fractional solution. In particular, $F \ge \mathrm{OPT}$, and the inequality can be strict, as in the example above.*

Proof. The idea is that the greedy fractional solution uses all of the space, and uses it at least as profitably. Let S be any feasible solution. Suppose S fills ℓ units of the knapsack with items not packed by the greedy fractional solution. Since the greedy fractional solution fills the knapsack completely, it fills at least ℓ units with items not packed by S. By the greedy ordering, every item of the second kind has bang-per-buck at least that of every item of the first kind, so the exchange does not lose value.

To make this precise, let x_i be the fraction of item i in the greedy fractional solution, let $y_i = 1$ if $i \in S$ and 0 otherwise, and let $r = v_{k+1}/w_{k+1}$. If $x_i > y_i$, then $i \le k+1$ and $v_i/w_i \ge r$; if $x_i < y_i$, then $i \ge k+1$ and $v_i/w_i \le r$. In both cases $(x_i - y_i)v_i \ge r(x_i - y_i)w_i$. Summing,

$$F - \sum_{i \in S} v_i = \sum_{i=1}^{n} (x_i - y_i) v_i \ge r\Big(W - \sum_{i \in S} w_i\Big) \ge 0. \qquad \square$$

Proof of Theorem 18.2. If Step 2 packs every item, the greedy solution is optimal. Otherwise, by Step 3, the value G of the three-step greedy solution is at least the total value of the first k items, and also at least the value v_{k+1} of item $k+1$. Adding these inequalities and using Lemma 18.3,

$$2G \geq \sum_{i=1}^{k+1} v_i \geq F \geq \text{OPT}.$$

□

Example: The analysis is tight. Let $W = 1000$, $(v_1, w_1) = (502, 501)$, and $(v_2, w_2) = (v_3, w_3) = (500, 500)$. Item 1 has the largest bang-per-buck, after which item 2 no longer fits, so the three-step greedy solution has value 502. The optimal solution $\{2, 3\}$ has value 1000.

The guarantee improves when every item is small relative to the capacity.

Proposition 18.4 [Small items]. *Let $0 < \delta < 1$. If $w_i \leq \delta W$ for every item i, then the solution from Steps 1 and 2 alone has value at least $(1-\delta) \cdot \text{OPT}$. For example, if every item has size at most 10% of W, the greedy value is at least 90% of optimal.*

Proof. If Step 2 packs every item, the solution is optimal. Otherwise item $k+1$ did not fit, so the first k items have total size $W_k > W - w_{k+1} \geq (1-\delta)W$; the knapsack is at least $(1-\delta)$ full. Let V_k be their total value. Each of them has bang-per-buck at least $r = v_{k+1}/w_{k+1}$, so $V_k \geq rW_k$. The fractional part of the greedy fractional solution occupies $W - W_k$ units at rate r. Hence

$$F = V_k + r(W - W_k) \leq V_k + \frac{V_k}{W_k}(W - W_k) = V_k \frac{W}{W_k} \leq \frac{V_k}{1-\delta}.$$

Therefore $V_k \geq (1-\delta)F \geq (1-\delta) \cdot \text{OPT}$ by Lemma 18.3. □

18.3 A dynamic programming heuristic

The goal is now an arbitrarily good approximation: for a user-specified parameter $\epsilon > 0$, such as $\epsilon = 0.01$, guarantee a $(1-\epsilon)$-approximation, a feasible solution whose value is at least $(1-\epsilon) \cdot \text{OPT}$. The catch is that the running time increases as ϵ decreases, so the algorithm exports a trade-off between running time and accuracy. This is the best-case scenario for an NP-complete problem.

The approach is to solve exactly a slightly incorrect, but easier, knapsack instance. If the sizes w_i and capacity W are integers, the earlier dynamic program solves knapsack in $O(nW)$ time. Alternatively, if the values v_i are integers, a second dynamic program (below) solves it in $O(n^2 v_{\max})$ time, where $v_{\max} = \max_i v_i$. Thus if all values are small integers, polynomial in n, knapsack is already solvable in polynomial time. The plan is to throw out the lower-order bits of the values.

Algorithm 18.2: Dynamic programming knapsack heuristic

Input: Values $v_i > 0$, sizes $w_i \le W$, capacity W, and accuracy $\epsilon > 0$.
Output: A feasible subset of items.

$m \leftarrow \epsilon v_{\max}/n$ *// justified in the analysis below*
for each item i **do**
  $\hat{v}_i \leftarrow \lfloor v_i/m \rfloor$
return an optimal solution of the knapsack instance with values $\hat{v}_1, \dots, \hat{v}_n$, sizes $w_1, \dots, w_n$, and capacity W

Step 1 rounds each v_i down to the nearest multiple of m and divides by m, producing integers $\hat{v}_i$. A larger m throws out more information and so loses accuracy; the choice of m, which depends on ϵ, is justified below. Step 2 uses the second dynamic program, in time $O(n^2 \max_i \hat{v}_i)$. Since the sizes and capacity are unchanged, the output is a feasible solution of the original instance.

18.4 Dynamic programming for knapsack, revisited

Assume the values v_i are nonnegative integers. Any solution has value at most $nv_{\max}$, so only targets in that range matter.

Definition 18.5 [Value-indexed subproblems]. For $i = 0, 1, \dots, n$ and $x = 0, 1, \dots, nv_{\max}$, let $S_{i,x}$ be the minimum total size of a subset of the first i items with total value at least x, or $+\infty$ if no such subset exists.

For $i \ge 1$, an optimal subset for (i, x) either omits item i, or contains it and uses the other chosen items to reach the remaining value $x - v_i$. Thus

$$S_{i,x} = \min\bigl\{S_{i-1,x},\ w_i + S_{i-1,\max\{x-v_i,0\}}\bigr\},$$

where the second index is 0 when $v_i > x$, since item i then reaches the target alone.

Algorithm 18.3: Knapsack by minimum size for each value

Input: Integer values $v_i \ge 0$, sizes w_i, and capacity W.
Output: The optimal knapsack value.

$A[0,0] \leftarrow 0$; $A[0,x] \leftarrow +\infty$ for $x = 1, \dots, nv_{\max}$
for $i \leftarrow 1$ **to** n **do**
  for $x \leftarrow 0$ **to** $nv_{\max}$ **do**
    $A[i,x] \leftarrow \min\{A[i-1,x],\ w_i + A[i-1, \max\{x - v_i, 0\}]\}$
return the largest x such that $A[n,x] \le W$

The loops perform $O(n^2 v_{\max})$ iterations with $O(1)$ work each, and the final scan takes $O(nv_{\max})$ time, so the running time is $O(n^2 v_{\max})$. An optimal subset is recovered by tracing back through the table.

18.5 Analysis of the dynamic programming heuristic

The analysis first finds how large m can be while still guaranteeing a $(1-\epsilon)$-approximation, then substitutes this value into the running time. Since v_i is rounded down to the nearest multiple of m,

$$m\hat{v}_i \in [v_i - m,\, v_i] \qquad \text{for each item } i.$$

Theorem 18.6 [Dynamic programming heuristic]. *With $m = \epsilon v_{\max}/n$, the heuristic returns a solution of value at least $(1-\epsilon)\cdot \mathrm{OPT}$ in $O(n^3/\epsilon)$ time.*

Proof. Let S be the heuristic's solution and S^* an optimal solution of the original instance. Use three facts: (1) $v_i \geq m\hat{v}_i$; (2) $m\hat{v}_i \geq v_i - m$; and (3) $\sum_{i\in S}\hat{v}_i \geq \sum_{i\in S^*}\hat{v}_i$, since S is optimal for the rounded values. Then

$$\sum_{i\in S} v_i \overset{(1)}{\geq} m\sum_{i\in S}\hat{v}_i \overset{(3)}{\geq} m\sum_{i\in S^*}\hat{v}_i \overset{(2)}{\geq} \sum_{i\in S^*}(v_i - m) \geq \sum_{i\in S^*} v_i - mn,$$

because S^* contains at most n items. To guarantee $\sum_{i\in S} v_i \geq (1-\epsilon)\sum_{i\in S^*} v_i$, it suffices that $mn \leq \epsilon \sum_{i\in S^*} v_i$. The optimal value is unknown to the algorithm, but it is at least $v_{\max}$, because the most valuable item fits on its own. So $mn = \epsilon v_{\max}$, that is, $m = \epsilon v_{\max}/n$, suffices.
For the running time, every item satisfies

$$\hat{v}_i \leq \frac{v_i}{m} \leq \frac{v_{\max}}{m} = v_{\max}\cdot\frac{n}{\epsilon v_{\max}} = \frac{n}{\epsilon}.$$

The second dynamic program therefore takes $O(n^2 \max_i \hat{v}_i) = O(n^3/\epsilon)$ time, and rounding takes $O(n)$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Approximation Algorithms for NP-Complete Problems.

Lecture 19: Local Search Algorithms

Local search starts from some candidate solution and repeatedly moves to a better nearby solution until none exists. It applies to many NP-complete problems; the price is that the final solution is only locally optimal, so its quality and the number of moves both need analysis.

19.1 The maximum cut problem

Definition 19.1 [Maximum cut]. The input is an undirected graph $G = (V, E)$. A cut (A, B) is a partition of V into two nonempty sets, and an edge crosses the cut if it has one endpoint in each set. The goal is a cut maximizing the number of crossing edges.

The maximum cut problem is NP-complete. Bipartite graphs, which by definition have a cut that every edge crosses, are a tractable special case: breadth-first search finds such a cut in linear time.

Example: In the graph of Figure 38, every one of the 8 edges has one endpoint in $A = \{w, x\}$ and the other in $B = \{u, v, y, z\}$. The graph is bipartite, so the maximum cut has value 8.

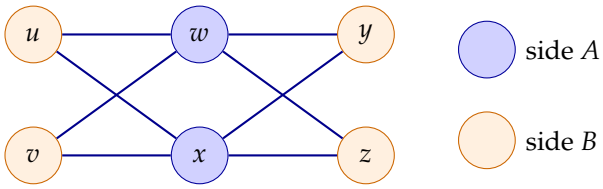


Figure 38: A bipartite graph on six vertices. The cut $A = \{w, x\}$, $B = \{u, v, y, z\}$ is crossed by all 8 edges, so it is a maximum cut.

For a cut (A, B) and a vertex v, let

$$c_v(A, B) = \#\text{ edges incident on } v \text{ that cross } (A, B),$$
$$d_v(A, B) = \#\text{ edges incident on } v \text{ that do not cross } (A, B).$$

Algorithm 19.1: Local search for maximum cut

Input: Undirected graph $G = (V, E)$.
Output: A locally optimal cut (A, B).

```
(A, B) ← an arbitrary cut of G
while some vertex v has d_v(A, B) > c_v(A, B) do
    move v to the other side of the cut
return (A, B)
```

Moving v to the other side changes the status of exactly the edges incident on v: its d_v non-crossing

edges start crossing and its c_v crossing edges stop. All other edges are unaffected. Each iteration therefore increases the number of crossing edges by $d_v(A,B) - c_v(A,B) \geq 1$. A simple graph has at most $\binom{n}{2}$ edges, where $n = |V|$, so there are at most $\binom{n}{2}$ iterations, and the algorithm runs in polynomial time. A cut on which the loop stops, so that $d_v(A,B) \leq c_v(A,B)$ for every vertex v, is LOCALLY OPTIMAL.

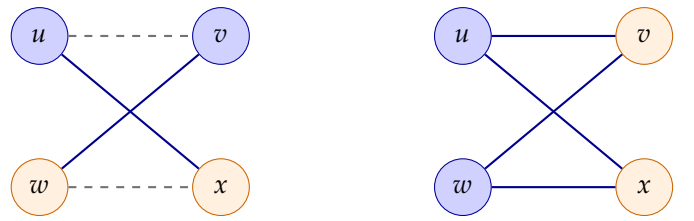
Theorem 19.2 [LOCAL SEARCH IS A $\frac{1}{2}$-APPROXIMATION]. *Every locally optimal cut is crossed by at least $\frac{1}{2}|E|$ edges. In particular, the local search algorithm outputs a cut with at least 50% of the maximum possible number of crossing edges.*

Proof. Let (A,B) be locally optimal, and sum $d_v(A,B) \leq c_v(A,B)$ over all $v \in V$. The left sum counts each non-crossing edge twice, once from each endpoint, and the right sum counts each crossing edge twice. Hence

$$2 \cdot [\#\text{ non-crossing edges}] \leq 2 \cdot [\#\text{ crossing edges}].$$

Adding $2 \cdot [\#\text{ crossing edges}]$ to both sides gives $2|E| \leq 4 \cdot [\#\text{ crossing edges}]$, so at least $\frac{1}{2}|E|$ edges cross. No cut has more than $|E|$ crossing edges. □

The factor $\frac{1}{2}$ is tight. In the four-cycle of Figure 39, the cut $A = \{u,v\}$, $B = \{w,x\}$ is crossed by 2 edges and every vertex has $c_v = d_v = 1$, so no single move helps. The cut $A^* = \{u,w\}$, $B^* = \{v,x\}$ is crossed by all 4 edges.



$A = \{u,v\}$, $B = \{w,x\}$ — 2 crossing edges

$A^* = \{u,w\}$, $B^* = \{v,x\}$ — 4 crossing edges

Figure 39: Tight example for Theorem 19.2. Solid edges cross the cut and dashed edges do not. Left: a locally optimal cut with 2 crossing edges. Right: a maximum cut with 4 crossing edges.

Note. The guarantee is no better than that of a random cut. Put each vertex in A or B independently with probability $\frac{1}{2}$, and for $e \in E$ let $X_e = 1$ if e crosses (A,B) and $X_e = 0$ otherwise. The two endpoints of e land on different sides with probability $\frac{1}{2}$, so $\mathbb{E}[X_e] = \mathbb{P}[X_e = 1] = \frac{1}{2}$. By linearity of expectation,

$$\mathbb{E}[\#\text{ crossing edges}] = \mathbb{E}\Big[\sum_{e \in E} X_e\Big] = \sum_{e \in E} \mathbb{E}[X_e] = \tfrac{1}{2}|E|.$$

In the WEIGHTED MAXIMUM CUT problem, each edge e has a nonnegative weight w_e, and the goal is to

maximize the total weight of crossing edges. Local search is still well defined: move v when the total weight of its non-crossing incident edges exceeds that of its crossing incident edges. The 50% guarantee still holds for locally optimal cuts, by the same double-counting argument with every count replaced by a total weight, and it also holds in expectation for a random cut. However, local search is no longer guaranteed to converge in polynomial time.

19.2 Principles of local search

Let X be the set of candidate solutions to a problem, such as the cuts of a graph, the tours of a traveling salesman instance, or the variable assignments of a constraint satisfaction problem. The key design choice is a NEIGHBORHOOD: for each $x \in X$, specify which $y \in X$ are its neighbors. For example:

- two cuts are neighbors if they differ by moving one vertex;
- two variable assignments are neighbors if they differ in the value of a single variable;
- two traveling salesman tours are neighbors if they differ in exactly two edges (Figure 40).

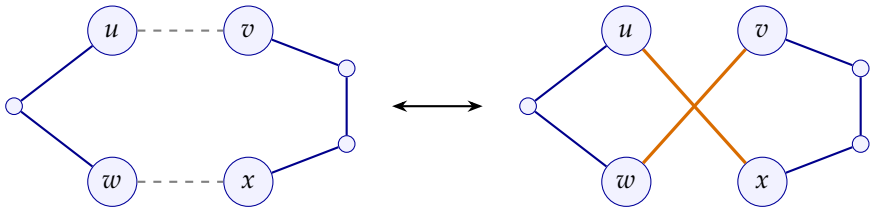


Figure 40: Neighboring tours. Removing the tour edges (u, v) and (w, x) and adding (u, x) and (w, v) gives another tour.

Algorithm 19.2: Generic local search

Input: A set X of candidate solutions with neighborhoods and an objective.
Output: A locally optimal solution.

```
x ← some initial solution
while x has a neighbor y that is superior to x do
    x ← y
return x
```

The generic algorithm leaves three choices open.

- *Initial solution.* Use a random solution, run many independent trials, and return the best local optimum found. Alternatively, start from the output of a good heuristic and use local search as a postprocessing step that can only improve it.
- *Which superior neighbor.* Choose one at random, choose the one with the biggest improvement, or use a more complex heuristic.

- *Neighborhoods.* Bigger neighborhoods make local optimality slower to verify but leave fewer bad local optima. The aim is a balance between solution quality and efficient searchability.

If X is finite and every step strictly improves some objective function, local search terminates, since no solution can repeat. It is usually not guaranteed to converge quickly, although it often does in practice; smoothed analysis is one way to explain this. Locally optimal solutions are generally not good approximations to global optima; running randomized local search many times and keeping the best local optimum mitigates this.

19.3 The 2-SAT problem

Definition 19.3 [2-SAT]. The input is n Boolean variables $x_1, \dots, x_n$ and m clauses, each a disjunction of 2 LITERALS, where a literal is some x_i or $\neg x_i$. The output is "yes" if some assignment of true and false values satisfies every clause simultaneously, and "no" otherwise.

Example: The instance $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee x_4) \wedge (\neg x_2 \vee \neg x_4)$ is a "yes" instance, satisfied for example by $x_1 = x_3 = \text{true}$ and $x_2 = x_4 = \text{false}$.

2-SAT can be solved in polynomial time, for example by a reduction to computing strongly connected components, by backtracking, or by the randomized local search below. In contrast, 3-SAT, where clauses have three literals, is the canonical NP-complete problem. Brute-force search takes about 2^n time, and randomized local search achieves about $(4/3)^n$ time (Schöning, 2002).

Algorithm 19.3: Papadimitriou's 2-SAT algorithm

```
Input: A 2-SAT instance with n variables.
Output: A satisfying assignment, or "unsatisfiable".
for ⌈log_2 n⌉ outer iterations do
    choose a uniformly random initial assignment
    for 2n^2 inner iterations do
        if the current assignment satisfies every clause then return it
        pick an arbitrary unsatisfied clause
        flip the value of one of its two variables, chosen uniformly at random
return "unsatisfiable"
```

The algorithm runs in polynomial time, since it performs $O(n^2 \log n)$ inner iterations, each of which checks the m clauses. It is always correct on unsatisfiable instances, because it reports an assignment only after verifying it. The key question is whether, on a satisfiable instance, it finds a satisfying assignment with probability close to 1. The analysis rests on random walks.

19.4 Random walks on a line

Consider a walk on the nonnegative integers that starts at position 0 at time 0. At each time step, the position goes up or down by 1 with probability $\frac{1}{2}$ each, except that from position 0 it moves to

position 1 with probability 1. For an integer $n \geq 1$, let T_n be the number of steps until the walk first reaches position n. It is a random variable over the coin flips at all time steps.

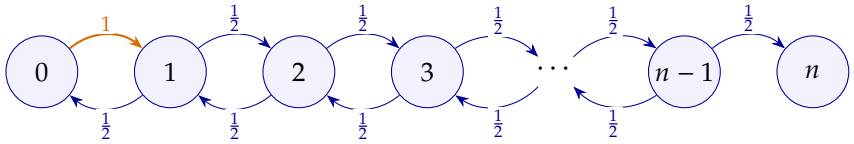


Figure 41: The random walk on the nonnegative integers. Position 0 moves to 1 with probability 1; every other position moves left or right with probability $\frac{1}{2}$ each.

Theorem 19.4 [Expected hitting time]. $\mathbb{E}[T_n] = n^2$.

Proof. For $0 \leq i \leq n$, let Z_i be the number of steps needed to reach n starting from position i, so $Z_0 = T_n$. At the edges,

$$\mathbb{E}[Z_n] = 0, \qquad \mathbb{E}[Z_0] = 1 + \mathbb{E}[Z_1].$$

For $1 \leq i \leq n-1$, condition on the first step. It goes left or right with probability $\frac{1}{2}$ each, after which the number of remaining steps is distributed as Z_{i-1} or Z_{i+1}, respectively:

$$\mathbb{E}[Z_i] = \tfrac{1}{2}\big(1 + \mathbb{E}[Z_{i-1}]\big) + \tfrac{1}{2}\big(1 + \mathbb{E}[Z_{i+1}]\big) = 1 + \tfrac{1}{2}\mathbb{E}[Z_{i+1}] + \tfrac{1}{2}\mathbb{E}[Z_{i-1}].$$

Rearranging,

$$\mathbb{E}[Z_i] - \mathbb{E}[Z_{i+1}] = \mathbb{E}[Z_{i-1}] - \mathbb{E}[Z_i] + 2.$$

The first difference is $\mathbb{E}[Z_0] - \mathbb{E}[Z_1] = 1$, so the consecutive differences are $1, 3, 5, \ldots$, and $\mathbb{E}[Z_i] - \mathbb{E}[Z_{i+1}] = 2i + 1$. Summing the differences for $i = 0, \ldots, n-1$ telescopes:

$$\mathbb{E}[Z_0] - \mathbb{E}[Z_n] = \sum_{i=0}^{n-1} (2i + 1) = n^2,$$

since pairing the term $2i + 1$ with $2(n - 1 - i) + 1$ gives pairs summing to $2n$, and there are $n/2$ such pairs. Since $\mathbb{E}[Z_n] = 0$, this gives $\mathbb{E}[T_n] = \mathbb{E}[Z_0] = n^2$. □

Corollary 19.5. $\mathbb{P}[T_n > 2n^2] \leq \frac{1}{2}$.

Proof. This is a special case of Markov's inequality. Let $p = \mathbb{P}[T_n > 2n^2]$. Split the expectation at $2n^2$; the first part is nonnegative, and every k in the second part is at least $2n^2$:

$$n^2 = \mathbb{E}[T_n] = \sum_{k=0}^{2n^2} k\,\mathbb{P}[T_n = k] + \sum_{k=2n^2+1}^{\infty} k\,\mathbb{P}[T_n = k] \geq 2n^2\,\mathbb{P}[T_n > 2n^2] = 2n^2 p.$$

Therefore $p \leq \frac{1}{2}$. □

19.5 Analysis of Papadimitriou's algorithm

Theorem 19.6 [Papadimitriou's algorithm on satisfiable instances]. *For a satisfiable 2-SAT instance with n variables, Papadimitriou's algorithm produces a satisfying assignment with probability at least $1 - \frac{1}{n}$.*

Proof. First consider a single outer iteration. Fix an arbitrary satisfying assignment a^*. Let a_t be the algorithm's assignment after inner iteration t, for $t = 0, 1, \ldots, 2n^2$, and let $X_t \in \{0, 1, \ldots, n\}$ be the number of variables on which a_t and a^* agree. Both are random variables. If $X_t = n$, then $a_t = a^*$ and the algorithm halts with a satisfying assignment.

Suppose a_t is not satisfying and the algorithm picks an unsatisfied clause on variables x_i and x_j. Since a_t makes both literals of the clause false while a^* makes at least one true, a^* differs from a_t on x_i, on x_j, or on both. Flipping a random one of the two gives:

1. if a^* and a_t differ on both x_i and x_j, then $X_{t+1} = X_t + 1$ with probability 1;
2. if they differ on exactly one of x_i, x_j, then $X_{t+1} = X_t + 1$ or $X_t - 1$ with probability $\frac{1}{2}$ each.

In particular, if $X_t = 0$ then a_t disagrees with a^* everywhere and case 1 applies, matching the walk's forced move from 0.

So $X_0, X_1, \ldots, X_{2n^2}$ behave like the random walk on the nonnegative integers, with three differences, all of which only help: some steps move right with probability 1 instead of $\frac{1}{2}$; X_0 may exceed 0; and the process may stop early at a satisfying assignment other than a^*, before $X_t = n$. Hence a single outer iteration finds a satisfying assignment with probability at least

$$\mathbb{P}[T_n \le 2n^2] \ge \tfrac{1}{2},$$

by Corollary 19.5. The outer iterations use fresh random choices, so they are independent, and

$$\mathbb{P}[\text{algorithm fails}] \le \mathbb{P}[\text{all } \lceil \log_2 n \rceil \text{ outer iterations fail}] \le \left(\tfrac{1}{2}\right)^{\lceil \log_2 n \rceil} \le \frac{1}{n}. \qquad \square$$

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Local Search Algorithms.

Lecture 20: The Wider World of Algorithms

This unit surveys problems and models beyond the course: matchings, flows, linear programming, and several other directions.

20.1 Stable matchings

Consider two node sets U and V, thought of as "men" and "women", with $|U| = |V| = n$ for simplicity. Each node ranks all nodes on the other side, and different nodes may have different rankings. Applications include matching hospitals with residents and colleges with applicants.

Definition 20.1 [Stable matching]. A perfect matching matches each node of U to a distinct node of V. It is a stable matching if, whenever $u \in U$ and $v \in V$ are not matched to each other, either u prefers his mate v' to v, or v prefers her mate u' to u. That is, no unmatched pair would both rather be with each other than with their mates.

Algorithm 20.1: Gale–Shapley proposal algorithm

Input: Sets U, V of size n, each node with a ranking of the other side.
Output: A stable matching.

```
while some man u is unattached do
    u proposes to the top woman v on his list who has not yet rejected him
    v keeps the best proposal received so far and rejects the other
return the current engagements
```

Each woman holds at most one proposal and each man is engaged to at most one woman, so throughout the algorithm the current engagements form a matching. A woman who has received a proposal stays engaged from then on, and each change of her partner is to a man she prefers.

Example: Let every man A, B, C rank the women D, E, F in that order, while D ranks A, B, C, E ranks B, C, A, and F ranks C, A, B. All three men first propose to D, who keeps A. Then B proposes to E and is accepted; C proposes to E, who prefers B; finally C proposes to F. The result is $\{(A, D), (B, E), (C, F)\}$ (Figure 42).

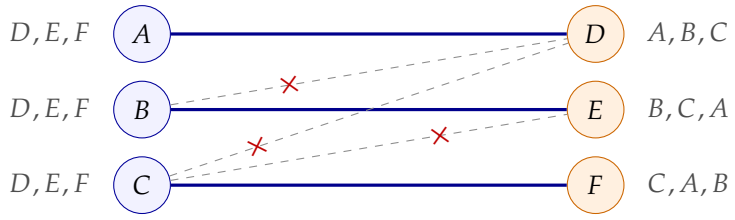


Figure 42: The proposal algorithm on the example. Preference lists are written beside each node. Rejected proposals are crossed out; the remaining proposals form the stable matching.

Theorem 20.2 [Gale–Shapley]. *The proposal algorithm terminates with a stable matching after at most n^2 iterations. In particular, a stable matching always exists.*

Proof. Running time. Each iteration is one proposal, and each man proposes to each woman at most once, so each man makes at most n proposals and there are at most n^2 iterations.
Perfect matching. Suppose some man is rejected by all n women. Then every woman has received a proposal, so all n women are engaged at the end. Since the engagements form a matching, all n men are engaged as well, a contradiction. So the algorithm ends only when no man is unattached, and the final matching is perfect.
Stability. Consider $u \in U$ and $v \in V$ not matched to each other. If u never proposed to v, then, because u proposes in order of preference, he is matched to someone he prefers to v. If u did propose to v, then v rejected him for a better offer at some point; since women only trade up, v ends matched to someone she prefers to u. □

20.2 Bipartite matching and maximum flow

Definition 20.3 [Bipartite matching]. The input is a bipartite graph $G = (U, V, E)$, so each edge has one endpoint in U and one in V. A matching is a set $M \subseteq E$ of pairwise disjoint edges. The goal is a matching of maximum size.

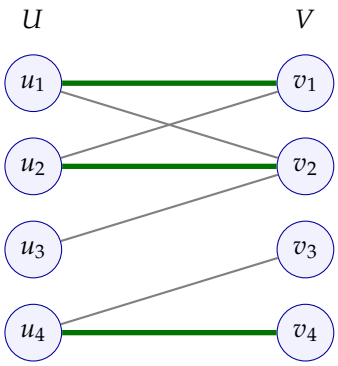


Figure 43: A bipartite graph whose maximum matching (thick edges) has size 3. The left nodes u_1, u_2, u_3 have only the two neighbors v_1, v_2, so at most two of them can be matched.

There is a straightforward reduction from bipartite matching to the maximum flow problem.

Definition 20.4 [Maximum flow]. The input is a directed graph $G = (V, E)$ with a source vertex s, a sink vertex t, and a capacity u_e on each edge e. An s-t flow assigns each edge an amount between 0 and its capacity such that, at every vertex other than s and t, the flow in equals the flow out. The goal is to send as much flow as possible from s to t.

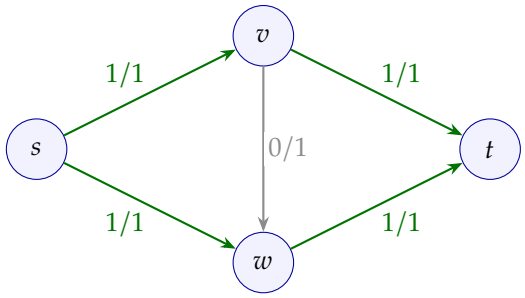


Figure 44: A flow network with unit capacities. Edge labels give flow/capacity for a maximum flow of value 2, which uses the paths $s \to v \to t$ and $s \to w \to t$.

The maximum flow problem is solvable in polynomial time, for example by nontrivial greedy algorithms based on "augmenting paths". They are nontrivial because a naive greedy choice can get stuck: in Figure 44, sending one unit along $s \to v \to w \to t$ leaves no s-t path with spare capacity, although the maximum flow value is 2.

20.3 Selfish flow and Braess's paradox

In a SELFISH FLOW model, a flow network carries one unit of selfish traffic, and each edge has a DELAY FUNCTION giving its travel time as a function of the load on the edge. Each part of the traffic uses a path of minimum total delay.

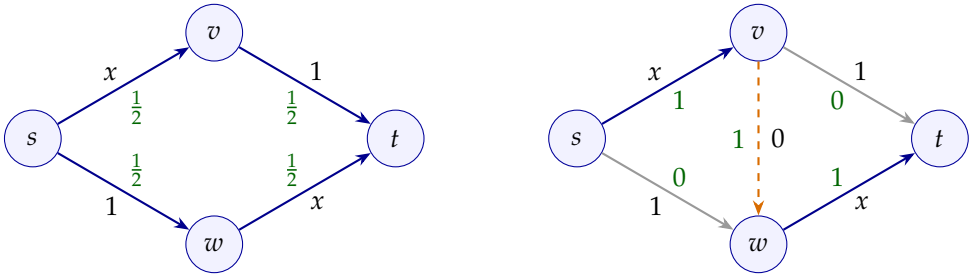


Figure 45: Braess's paradox. Black labels give each edge's delay as a function of its load x; green labels give the load. Left: the two routes have equal delay, and traffic splits evenly, so each route takes $\frac{1}{2} + 1 = 1.5$. Right: after adding a zero-delay edge from v to w, all traffic takes $s \to v \to w \to t$, which takes $1 + 0 + 1 = 2$.

In the network of Figure 45, the edges (s, v) and (w, t) have delay x under load x, while (v, t) and (s, w) have constant delay 1. In the steady state, traffic splits 50/50, and the commute time is 1.5 hours on either route. BRAESS'S PARADOX (1968): after adding a zero-delay "teleporter" from v to w, the commute time of selfish traffic degrades to 2 hours. Indeed, when all traffic uses $s \to v \to w \to t$, each of (s, v) and (w, t) carries load 1; every route then has delay 2, so no traffic can improve by switching.

20.4 Linear programming

The general LINEAR PROGRAMMING problem is to optimize a linear function over the intersection of halfspaces. It generalizes maximum flow and many other problems. Linear programs can be solved efficiently, both in theory and in practice, which makes linear programming a very powerful "black-box" subroutine.

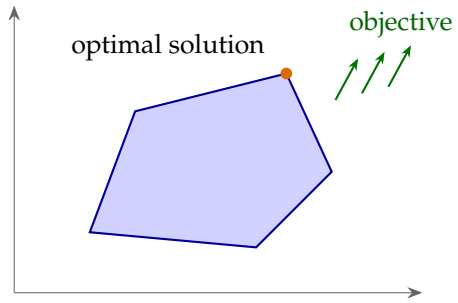


Figure 46: Schematic linear program in two dimensions. The feasible region is an intersection of halfplanes; the optimum is a point of the region that is furthest in the direction of the objective function.

Two extensions are CONVEX PROGRAMMING, which is polynomial-time solvable under mild conditions, and INTEGER PROGRAMMING, which is NP-hard in general.

20.5 Other topics and models

- Deeper study of data structures, graph algorithms, approximation algorithms, and other course topics.
- Geometric algorithms, both low-dimensional (for example, convex hull) and high-dimensional (for example, nearest neighbors in information retrieval).
- Algorithms that run forever, usually in real time, such as caching and routing.
- Algorithms with bounded memory, or STREAMING ALGORITHMS, for example to maintain statistics at a network router.
- Algorithms that exploit parallelism, for example via MapReduce and Hadoop.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), The Wider World of Algorithms.